

## Full Length Article



# A novel singularity- and discontinuity-capturing PINN for time-fractional diffusion equations involving initial singularities and interfaces on complex curved surfaces

Hongji Li , Zhijun Tan \*

School of Computer Science and Engineering, Guangdong Province Key Lab of Computational Science, Sun Yat-sen University, 510006, Guangzhou, Guangdong, China

## ARTICLE INFO

**Keywords:**

Singularity- and discontinuity-capturing PINN (SDC-PINN)  
Interface PDEs  
Complex curved surfaces  
Time-fractional diffusion equations  
Initial singularity  
Nonuniform  $L2-1_\sigma$  scheme

## ABSTRACT

In this paper, we propose a novel singularity- and discontinuity-capturing physics-informed neural network (SDC-PINN) designed to tackle challenging time-fractional diffusion problems characterized by initial singularities and interfaces on complex curved surfaces. The SDC-PINN method incorporates a singularity-capturing feature in the temporal direction at the initial time, alongside a discontinuity-capturing feature in the spatial direction within the neural network input. This approach preserves the inherent properties of the solution, effectively and accurately capturing the solution's sharpness as well as that of its derivatives at the interface, while adeptly resolving the initial singularity. To further address the initial singularity of the solution, we employ the classical nonuniform  $L2-1_\sigma$  scheme in time to approximate the Caputo fractional derivative of the neural network output. Additionally, the proposed network accommodates the use of scattered training points, thereby facilitating the efficient handling of problems on more complex curved surfaces. Numerous numerical experiments are performed to assess the effectiveness and precision of the proposed SDC-PINN.

## 1. Introduction

Fractional partial differential equations (FPDEs) are distinguished by their spatial heterogeneity and non-local characteristics. Due to these features, FPDEs have been widely employed to model and analyze numerous significant phenomena and dynamic processes across multiple disciplines, such as physics, biology, chemistry, and economics. Examples include solute transportation in fractured and porous media [1], signal and image frequency characteristics [28], and viscoelastic constitutive relationships [15], to name just a few. Although FPDEs have a wide range of applications, deriving analytical solutions to them is frequently difficult, as the exact solutions typically involve special functions. This has led to a significant necessity for the advancement of numerical methods that can effectively approximate these solutions, thus realizing the full potential of FPDEs in varied fields. In recent decades, noteworthy advancements have been achieved in the numerical solution of FPDEs, resulting in the development of diverse techniques, such as the finite difference method [6,12], finite element method [3,5], spectral method [2,29], among others.

In recent times, the swift progress in deep learning technologies has sparked interest in utilizing neural networks to tackle partial differential equations (PDEs). Within this field, the physics-informed neural networks (PINNs) [18] stands out as one of the most promising frameworks. PINNs integrates the observational data and physical laws into the neural network's training process, thereby

\* Corresponding author.

E-mail address: [tzhi@jmail.sysu.edu.cn](mailto:tzhi@jmail.sysu.edu.cn) (Z. Tan).

effectively reformulating the problems of solving PDEs into optimization tasks. A key characteristic of PINNs is its inherent mesh-free nature, allowing it to tackle the computational challenges related to the "curse of dimensionality" and the difficulties presented by intricate boundaries.

The effectiveness of PINNs depends on utilizing automatic differentiation to derive integer-order derivatives of neural network outputs analytically. However, this approach faces challenges when used with FPDEs, primarily due to the inapplicability of the classical chain rule within fractional calculus. To tackle this issue, fractional PINNs (fPINNs) [16] utilizes conventional numerical techniques to discretize fractional operators, which are then integrated into the loss function of PINNs. Expanding on fPINNs, Ren et al. introduced improved fractional PINNs (IFPINNs) [19] to address time-fractional differential equations. IFPINNs leverages the nonuniform L1 scheme to estimate the Caputo fractional derivative of neural network output, substituting linear functions with non-linear ones throughout the weight propagation process between layers. This modification significantly boosts the capacity of the neural networks to handle problems associated with high oscillations or singular perturbations. Conventional methods for estimating fractional derivatives become impractical for high-dimensional FPDEs owing to excessive computational demands as physical dimensions increase. To mitigate this challenge, Monte Carlo sampling-based PINNs (MC-fPINNs) [7] treats the fractional derivative as the expectation of a function. It is worth noting that there is relatively little work on neural networks for handling the FPDEs with initial singularities, and most existing neural network methods assume that the solution is sufficiently smooth at the initial time.

In the natural world, a variety of problems involve multiple distinct materials or fluids that exhibit different conductivities, densities, or diffusion coefficients. This situation often results in the formation of interfaces and the development of equations characterized by discontinuous coefficients across these interfaces, commonly termed interface PDEs. Recent developments in neural network techniques have demonstrated the potential for effectively tackling these interface PDEs. Primarily, two neural network strategies have been introduced. The first strategy utilizes a domain decomposition-based neural network technique [8,20,21,27,33], which employs different neural networks for distinct subdomains divided by the sharp interface. The second strategy employs the discontinuity-capturing method [10,26,32], where an auxiliary feature correlated with the spatial variable is incorporated into the neural network's input. This approach enables utilizing one neural network to maintain the essential solution characteristics of interface PDEs, including both the sharpness of the solution and that of its derivatives at the interface. Recently, we proposed a discontinuity-removing PINNs (DR-PINNs) [30,31] which utilizes a regularity decomposition strategy, significantly mitigating errors in the vicinity of the interface.

The objective of this work is to develop an efficient neural network method for addressing time-fractional diffusion problems that involve both initial singularities and interfaces on complex curved surfaces. Notably, significant advancements have been made in tackling single-aspect interface PDEs and single-aspect FPDEs with initial smoothness on flat surfaces through the application of neural network techniques. However, research on the use of neural networks to address single-aspect FPDEs characterized by discontinuous coefficients and interfaces appears to be limited, even on flat surfaces. Furthermore, even traditional numerical research aimed at solving time-fractional diffusion problems with interfaces is also fairly scarce, with the exception of [4], primarily due to the complexities inherent in interface FPDEs. Additionally, the existing literature indicates that only [30] uses machine learning for tackling surface interface PDEs. Moreover, we observe a significant lack of studies applying neural networks to FPDEs with initial singularities, even on flat surfaces, let alone on curved ones.

Building upon the work of [10,16,19], this study's notable contribution is the introduction of a novel SDC-PINN framework. This framework is specifically designed to tackle time-fractional diffusion problems that exhibit both initial singularities and interfaces on complex curved surfaces, rather than being restricted to planar surfaces. Similar to the discontinuity-capturing technique, the proposed SDC-PINN method integrates both features for capturing discontinuities and initial singularities within the input of the neural network. This design allows the network to accurately recognize both the sharpness of the solution and that of its derivatives at the interface, as well as the initial singularity, a task that traditional neural networks often struggle to perform effectively. Furthermore, to enhance the handling of the initial singularity, we employ the nonuniform L2-1 <sub>$\sigma$</sub>  scheme to approximate the Caputo fractional derivative of the neural network output.

The remainder of this paper is organized as follows: In Section 2, we present time-fractional diffusion problems characterized by discontinuous coefficients and interfaces on complex curved surfaces, and we review the nonuniform L1 scheme and the nonuniform L2-1 <sub>$\sigma$</sub>  scheme. In Section 3, we discuss the SDC-PINN method developed to address these problems. Section 4 showcases the proposed algorithm's effectiveness and precision via a series of numerical tests. Finally, the findings and future work are presented in Section 5.

## 2. Preliminaries

### 2.1. The model problem

Anomalous diffusion on surfaces, which describes phenomena that do not adhere to Fick's law, is a topic of growing importance in the scientific community. These non-Fickian dynamics are frequently observed in complex environments such as porous media, turbulent fluids, plasmas, and biological systems [36,37]. Standard diffusion equations have proven insufficient to effectively characterize these anomalous processes. To address this limitation, fractional diffusion models have been proposed as a more suitable alternative to traditional nonlinear diffusion models. The advantage of fractional models lies in their parametric parsimony, as they can capture complex behavior with fewer, more accessible parameters. In this work, we focus on the following surface time-fractional diffusion problem with interface.

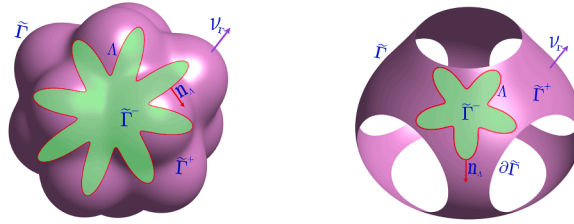


Fig. 1. An illustration of a closed surface (Left) and an open surface (Right), each with a smooth curve interface  $\Lambda$ .

Consider an oriented and compact surface  $\tilde{\Gamma}$ , represented by

$$\tilde{\Gamma} = \{\mathbf{x} \in \mathcal{N} | \phi(\mathbf{x}) = 0\}, \quad (2.1)$$

where  $\mathcal{N}$  is an open tube in three-dimensional Euclidean space, and  $\phi \in C^2(\mathcal{N})$  is a function satisfying  $\nabla\phi(\mathbf{x}) \neq 0$ . The interface  $\Lambda$ , a surface curve (assumed to be smooth and closed), separates  $\tilde{\Gamma}$  into  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$ , satisfying  $\tilde{\Gamma} = \tilde{\Gamma}^+ \cup \Lambda \cup \tilde{\Gamma}^-$ . We will now investigate the time-fractional diffusion problem with interface concerning an initial singularity defined on complex curved surface  $\tilde{\Gamma}$ ,

$$\begin{cases} \partial_t^\alpha u(\mathbf{x}, t) - \nabla_{\tilde{\Gamma}} \cdot (\beta \nabla_{\tilde{\Gamma}} u(\mathbf{x}, t)) + \kappa u(\mathbf{x}, t) = f(\mathbf{x}, t), & \mathbf{x} \in \tilde{\Gamma}^+ \cup \tilde{\Gamma}^-, t \in (0, T], \end{cases} \quad (2.2a)$$

$$\begin{cases} [u]_{\Lambda} = \lambda(\mathbf{x}_{\Lambda}, t), & \mathbf{x}_{\Lambda} \in \Lambda, t \in (0, T], \end{cases} \quad (2.2b)$$

$$\begin{cases} [\beta \nabla_{\tilde{\Gamma}} u \cdot n_{\Lambda}]_{\Lambda} = \rho(\mathbf{x}_{\Lambda}, t), & \mathbf{x}_{\Lambda} \in \Lambda, t \in (0, T], \end{cases} \quad (2.2c)$$

$$\begin{cases} u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \tilde{\Gamma}^+ \cup \tilde{\Gamma}^-. \end{cases} \quad (2.2d)$$

A Dirichlet boundary condition given by  $u = u_b$  for  $\mathbf{x} \in \partial\tilde{\Gamma}$  is applied when  $\tilde{\Gamma}$  represents an open surface. The vector  $n_{\Lambda}$ , tangential to the surface  $\tilde{\Gamma}$ , functions as the unit normal vector associated with  $\Lambda$ . It is directed towards  $\tilde{\Gamma}^+$  (refer to Fig. 1 for a visual depiction). The notation  $[\cdot]$  signifies the jump of a quantity, defined as the limiting value at the interface  $\Lambda$  from the  $\tilde{\Gamma}^+$  side subtracted by that on the  $\tilde{\Gamma}^-$  side, i.e.,

$$[u](\mathbf{x}_{\Lambda}, \cdot) = \lim_{\mathbf{x} \in \tilde{\Gamma}^+, \mathbf{x} \rightarrow \mathbf{x}_{\Lambda}} u(\mathbf{x}, \cdot) - \lim_{\mathbf{x} \in \tilde{\Gamma}^-, \mathbf{x} \rightarrow \mathbf{x}_{\Lambda}} u(\mathbf{x}, \cdot), \quad \mathbf{x}_{\Lambda} \in \Lambda. \quad (2.3)$$

Related to the jump conditions, the functions  $\lambda$  and  $\rho$  are defined as continuous on  $\Lambda$ . Additionally, the diffusion coefficient  $\beta = \beta(\mathbf{x}) > 0$  is a piecewise continuous function as follows:

$$\beta(\mathbf{x}) = \begin{cases} \beta^+(\mathbf{x}), & \mathbf{x} \in \tilde{\Gamma}^+, \\ \beta^-(\mathbf{x}), & \mathbf{x} \in \tilde{\Gamma}^-. \end{cases} \quad (2.4)$$

The reaction coefficient  $\kappa \geq 0$  is defined as a piecewise continuous function. The notation  $\partial_t^\alpha u$  represents the Caputo fractional derivative [13,14], which is given by

$$\partial_t^\alpha u := {}^C D_t^\alpha u = \frac{1}{\Gamma(1-\alpha)} \int_0^t (t-s)^{-\alpha} \frac{\partial u(\mathbf{x}, s)}{\partial s} ds, \quad 0 < \alpha < 1, \quad (2.5)$$

where  $\Gamma(\cdot)$  is the gamma function. Additionally, the symbol  $\nabla_{\tilde{\Gamma}}$  and  $\nabla_{\tilde{\Gamma}} \cdot$  denote the surface gradient and surface divergence operators respectively. At a given point  $\mathbf{x} \in \tilde{\Gamma}$ , we can reformulate these surface differential operators on  $\tilde{\Gamma}$  using the conventional differential operators in Euclidean spaces as follows:

$$\nabla_{\tilde{\Gamma}} u(\mathbf{x}) = (\mathbb{I} - v_{\tilde{\Gamma}} v_{\tilde{\Gamma}}^T) \nabla u(\mathbf{x}), \quad (2.6a)$$

$$\nabla_{\tilde{\Gamma}} \cdot u(\mathbf{x}) = \nabla \cdot u - v_{\tilde{\Gamma}}^T (\nabla u(\mathbf{x})) v_{\tilde{\Gamma}}. \quad (2.6b)$$

Here,  $v_{\tilde{\Gamma}} = v_{\tilde{\Gamma}}(\mathbf{x})$  is the unit normal vector on  $\tilde{\Gamma}$  and  $\mathbb{I}$  serves as the identity projection. The unit normal vector  $v_{\tilde{\Gamma}}$  can be computed via the formula  $v_{\tilde{\Gamma}} = \frac{\nabla\phi}{\|\nabla\phi\|}$ , where  $\|\cdot\|$  denotes the standard Euclidean norm. This research primarily concentrates on scenarios in which the typical solution  $u$  presents an initial weak singularity in the temporal direction [13,14], adhering to the following regularity condition for every  $t \in (0, T]$ :

$$|\partial_t^\ell u(\cdot, t)| \leq C(1 + t^{\alpha-\ell}), \quad \ell = 0, 1, 2, \quad (2.7)$$

where  $C$  is a positive constant. Eq. (2.7) indicates that the temporal derivatives of the solution are unbounded as  $t \rightarrow 0^+$ , i.e.  $\lim_{t \rightarrow 0^+} \partial_t u(\cdot, t) \rightarrow \infty$ .

## 2.2. Temporal discretization for fractional derivative

In recent decades, significant progress has been made by researchers focused on resolving initial singularities in the FPDEs. The primary numerical method used to tackle these initial singularities involves employing a nonuniform temporal mesh, such as a graded mesh [23]. Specifically, let  $N$  represent a positive integer, and consider the graded mesh defined by  $t_n = T(n/N)^r$  ( $n = 0, 1, \dots, N$ ).

Here,  $r$  signifies the exponent of the graded mesh. It is crucial to emphasize that when  $r = 1$ , the graded mesh becomes uniform. By employing this graded mesh approach, finite difference methods can effectively be utilized to discretize the fractional derivatives. Following this, we will explore two temporal discretizations related to the Caputo fractional derivative (2.5). Before proceeding, let us introduce the following essential notations:

$$\begin{aligned}\tau_n &= t_n - t_{n-1}, \quad \tau = \max_{1 \leq n \leq N} \tau_n, \quad u^n = u(\cdot, t_n), \\ \rho_n &= \tau_n / \tau_{n+1}, \quad t_{n-\sigma} = (1 - \sigma)t_n + \sigma t_{n-1}, \quad u^{n-\sigma} = u(\cdot, t_{n-\sigma}), \quad \sigma = \alpha/2.\end{aligned}$$

### 2.2.1. The L1 scheme

Based on the graded mesh for the time interval  $(0, T]$ , the Caputo fractional derivative (2.5) can be reformulated as

$${}_0^C D_t^\alpha u(\mathbf{x}, t_n) = \frac{1}{\Gamma(1-\alpha)} \sum_{k=0}^{n-1} \int_{t_k}^{t_{k+1}} (t_n - s)^{-\alpha} \frac{\partial u(\mathbf{x}, s)}{\partial s} ds.$$

For  $n \geq 1$ , the widely recognized L1 formula used for approximating the Caputo fractional derivative (2.5) is expressed by

$$\begin{aligned}{}_0^C D_t^\alpha u(\mathbf{x}, t_n) &\approx D_N^\alpha u^n := \frac{1}{\Gamma(1-\alpha)} \sum_{k=0}^{n-1} \frac{u^{k+1} - u^k}{\tau_{k+1}} \int_{t_k}^{t_{k+1}} \frac{ds}{(t_n - s)^\alpha} \\ &= \frac{1}{\Gamma(2-\alpha)} \sum_{k=0}^{n-1} \frac{u^{k+1} - u^k}{\tau_{k+1}} [(t_n - t_k)^{1-\alpha} - (t_n - t_{k+1})^{1-\alpha}] \\ &= \frac{1}{\Gamma(2-\alpha)} [a_{n,1} u^n - a_{n,n} u^0 - \sum_{k=1}^{n-1} u^{n-k} (a_{n,k} - a_{n,k+1})],\end{aligned}\quad (2.8)$$

where  $a_{n,k} := [(t_n - t_{n-k})^{1-\alpha} - (t_n - t_{n-k+1})^{1-\alpha}] / \tau_{n-k+1}$  for  $k = 1, \dots, n$ . Note that  $a_{n,1} = \tau_n^{-\alpha}$ . According to established findings, the aforementioned nonuniform L1 scheme achieves an optimal convergence rate of  $\mathcal{O}(\tau^{\min(r\alpha, 2-\alpha)})$  [13].

### 2.2.2. The L2-1 $_\sigma$ scheme

Next, we introduce the nonuniform L2-1 $_\sigma$  formula of Alikhanov [14]. For  $n = 1, 2, \dots, N$ , we define the following kernels:

$$\begin{aligned}\mathcal{A}_0^n &= \frac{1}{\tau_n \Gamma(1-\alpha)} \int_{t_{n-1}}^{t_{n-\sigma}} (t_{n-\sigma} - s)^{-\alpha} ds = \frac{(t_{n-\sigma} - t_{n-1})^{1-\alpha}}{\tau_n \Gamma(2-\alpha)} = \frac{(1-\sigma)^{1-\alpha} \tau_n^{-\alpha}}{\Gamma(2-\alpha)}, \\ \mathcal{A}_{n-k}^n &= \frac{1}{\tau_k \Gamma(1-\alpha)} \int_{t_{k-1}}^{t_k} (t_{n-\sigma} - s)^{-\alpha} ds = \frac{(t_{n-\sigma} - t_{k-1})^{1-\alpha} - (t_{n-\sigma} - t_k)^{1-\alpha}}{\tau_k \Gamma(2-\alpha)}, \\ \mathcal{B}_{n-k}^n &= \frac{2}{\tau_k (t_{k+1} - t_{k-1}) \Gamma(1-\alpha)} \int_{t_{k-1}}^{t_k} (t_{n-\sigma} - s)^{-\alpha} (s - t_{k-\frac{1}{2}}) ds \\ &= \frac{2}{\tau_k (\tau_k + \tau_{k+1}) \Gamma(2-\alpha)} \left( -\frac{\tau_k}{2} ((t_{n-\sigma} - t_{k-1})^{1-\alpha} + (t_{n-\sigma} - t_k)^{1-\alpha}) + \frac{(t_{n-\sigma} - t_{k-1})^{2-\alpha} - (t_{n-\sigma} - t_k)^{2-\alpha}}{2-\alpha} \right),\end{aligned}$$

where  $1 \leq k \leq n-1$ . Now we shall introduce the discrete kernels  $C_k^{(n,\alpha)}$ . When  $n = 1$ , we define  $C_0^{(1,\alpha)} = \mathcal{A}_0^1$ . For  $2 \leq n \leq N$ , we proceed to define

$$C_{n-k}^{(n,\alpha)} = \begin{cases} \mathcal{A}_0^n + \rho_{n-1} \mathcal{B}_1^n, & k = n, \\ \mathcal{A}_{n-k}^n + \rho_{k-1} \mathcal{B}_{n-k+1}^n - \mathcal{B}_{n-k}^n, & 2 \leq k \leq n-1, \\ \mathcal{A}_{n-1}^n - \mathcal{B}_{n-1}^n, & k = 1. \end{cases}$$

We then utilize the nonuniform L2-1 $_\sigma$  scheme to approximate the Caputo fractional derivative at the point  $t_{n-\sigma}$  by

$${}_0^C D_t^\alpha u(\mathbf{x}, t_{n-\sigma}) \approx \delta_{t_{n-\sigma}}^\alpha u := C_0^{(n,\alpha)} u^n - \sum_{k=1}^{n-1} (C_{n-k-1}^{(n,\alpha)} - C_{n-k}^{(n,\alpha)}) u^k - C_{n-1}^{(n,\alpha)} u^0, \quad 1 \leq n \leq N. \quad (2.9)$$

It is well known that an optimal convergence rate of  $\mathcal{O}(\tau^{\min(r\alpha, 2)})$  for the nonuniform L2-1 $_\sigma$  scheme can be achieved [14].

## 3. A singularity- and discontinuity-capturing PINN (SDC-PINN)

In this section, we will introduce the singularity- and discontinuity-capturing PINN (SDC-PINN) to address the time-fractional diffusion problem (2.2), which involves initial singularity and interface. The objective is to integrate the discontinuity-capturing feature  $\mathcal{Y}$  and the initial singularity-capturing feature  $\mathcal{Z}$  into the neural network input, thereby effectively capturing the initial singularity as well as singularities in the solution and/or its derivatives. The flowchart and network architecture of SDC-PINN are shown in Fig. 2.

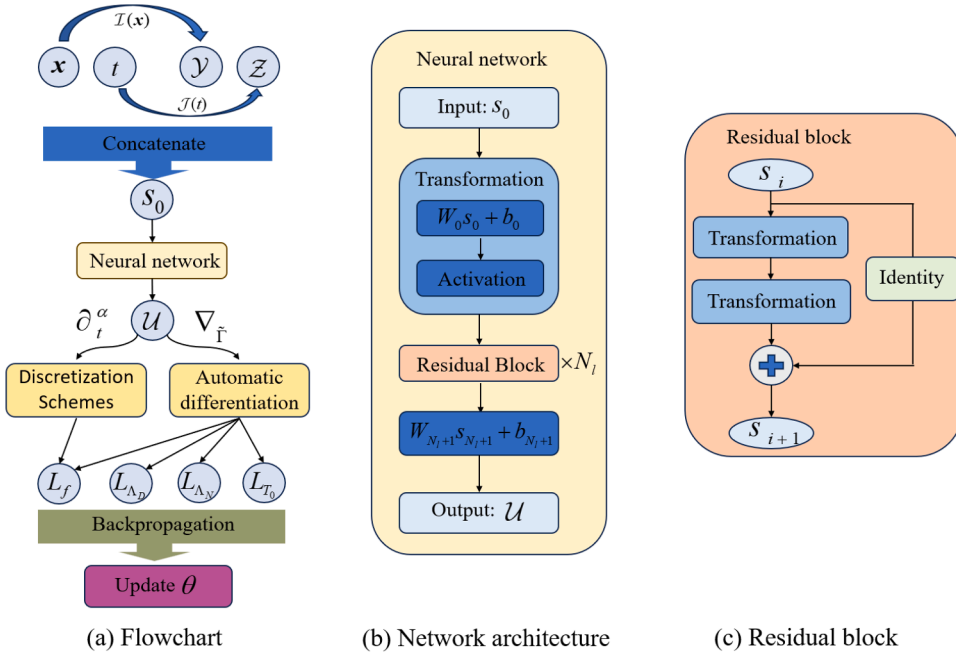


Fig. 2. The flowchart and network architecture of the SDC-PINN. The architecture incorporates residual blocks for improved training stability.

### 3.1. Discontinuity-capturing feature

Due to the presence of a discontinuity or jump in the solution  $u$  across the interface  $\Lambda$ , we utilize the extension-and-projection method presented in [10] and introduce an extra discontinuity-capturing feature  $\mathcal{Y} = \mathcal{I}(\mathbf{x})$  to represent each subdomain, where

$$\mathcal{I}(\mathbf{x}) = \begin{cases} 1, & \mathbf{x} \in \tilde{\Gamma}^+, \\ -1, & \mathbf{x} \in \tilde{\Gamma}^-. \end{cases} \quad (3.1)$$

Then, the solution  $u$  can be expressed using the neural network function  $\mathcal{U}(\mathbf{x}, t, \mathcal{Y}, \mathcal{Z})$ . The solutions  $u$  in  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$  can be interpreted as the projections of  $\mathcal{U}$  onto two separate hyperplanes (namely  $S^+ = \{(\mathbf{x}, t, 1) | \mathbf{x} \in \tilde{\Gamma}^+\}$  and  $S^- = \{(\mathbf{x}, t, -1) | \mathbf{x} \in \tilde{\Gamma}^-\}$ ). As a result, the jump condition in (2.2b) can be rephrased in terms of the differences between the projections of the continuous neural network function  $\mathcal{U}$  onto the two distinct hyperplanes  $S^+$  and  $S^-$ . Thus, for any  $\mathbf{x}_\Lambda \in \Lambda$ , one has

$$[u](\mathbf{x}_\Lambda, t) = \mathcal{U}(\mathbf{x}_\Lambda, t, 1, \mathcal{Z}) - \mathcal{U}(\mathbf{x}_\Lambda, t, -1, \mathcal{Z}). \quad (3.2)$$

Additionally, the jump condition in (2.2c) can be expressed in the following manner:

$$[\beta \nabla_{\tilde{\Gamma}} u \cdot \mathbf{n}_\Lambda](\mathbf{x}_\Lambda, t) = (\beta^+(\mathbf{x}_\Lambda) \nabla_{\tilde{\Gamma}} \mathcal{U}(\mathbf{x}_\Lambda, t, 1, \mathcal{Z}) - \beta^-(\mathbf{x}_\Lambda) \nabla_{\tilde{\Gamma}} \mathcal{U}(\mathbf{x}_\Lambda, t, -1, \mathcal{Z})) \cdot \mathbf{n}_\Lambda. \quad (3.3)$$

### 3.2. Initial singularity-capturing feature

As previously mentioned, this study primarily addresses the scenario where the solution to problem (2.2) exhibits an initial singularity. The universal approximation theorems [11] confirm the capability of neural networks to approximate such continuous solutions. However, due to their differentiable activation functions, neural networks inherently exhibit smoothness, which makes them insufficient for accurately representing solutions that have a singularity at  $t = 0$ . More specifically, fitting a singular derivative in neural network solutions presents a considerable challenge. Inspired by [19,26], we propose an additional variable  $\mathcal{Z} = \mathcal{J}(t)$  to effectively represent the characteristic of the initial singularity, where

$$\mathcal{J}(t) = t^{\xi} = t^{e^{\tilde{\xi}}}, \quad \tilde{\xi} \in \mathbb{R}^{N_{\xi}}. \quad (3.4)$$

It is important to note that  $\tilde{\xi}$  serves as a trainable vector parameter, with  $N_{\xi}$  set to 1 by default. This configuration adequately supports the approximation of solutions that have an initial singularity. Here, we refer to  $\mathcal{Z}$  as the feature that captures the initial singularity, as its derivative  $\frac{d\mathcal{Z}}{dt} = \xi t^{\xi-1}$  reveals a singularity at  $t = 0$  when there exists a  $\xi \in \tilde{\xi}$  such that  $\xi < 1$ . Based on this property, it is evident that the partial derivative of  $\mathcal{U}$  with respect to  $t$ ,

$$\frac{\partial u}{\partial t} = \frac{\partial \mathcal{U}}{\partial t} + \frac{\partial \mathcal{U}}{\partial \mathcal{Z}} \frac{d\mathcal{Z}}{dt} = \frac{\partial \mathcal{U}}{\partial t} + \frac{\partial \mathcal{U}}{\partial \mathcal{Z}} \xi t^{\xi-1}, \quad (3.5)$$

exhibits an initial singularity when  $\xi < 1$ . Notably, despite the addition of the singularity capturing feature  $\mathcal{Z}$  of the solution  $\mathcal{U}$ , there remains the possibility that its temporal derivative can be learned to a non-singularity state. For example, one might observe that when  $\frac{\partial \mathcal{U}}{\partial \xi} = 0$  or for every  $\xi$  belonging to  $\xi$ ,  $\xi \geq 1$ , the solution's partial derivative regarding  $t$  might not demonstrate an initial singularity.

### 3.3. The SDC-PINN involving features of singularity and discontinuity

In this subsection, we utilize the previously mentioned neural network  $\mathcal{U}(\mathbf{x}, t, \mathcal{Y}, \mathcal{Z})$  to tackle problem (2.2). As outlined in [16], traditional PINNs cannot be directly employed to solve FPDEs due to the inapplicability of the classical chain rule in fractional calculus. To overcome this challenge, fPINNs [16] implements standard numerical techniques to replace fractional differential operators with their discrete numerical approximation. These discrete representations are then incorporated into the loss function of the PINNs. Building on the ideas from fPINNs [16] and with the goal of addressing the initial singularity, we adopt the nonuniform L2-1 <sub>$\sigma$</sub>  scheme (2.9) to discretize the Caputo fractional derivative (2.5). Specifically, for the training point  $(\mathbf{x}, t)$ , we generate the corresponding auxiliary points  $\{(\mathbf{x}, t_i) | i = 0, 1, \dots, N_t\}$ , where

$$t_{N_t} = \frac{t}{1 - \sigma + \sigma(\frac{N_t-1}{N_t})^r}, \quad t_i = t_{N_t}(\frac{i}{N_t})^r, \quad i = 0, 1, \dots, N_t. \quad (3.6)$$

Here,  $N_t$  serves as a hyperparameter. The primary advantage of utilizing auxiliary points lies in their ability to accommodate scattered training points for the temporal mesh, thereby offering enhanced flexibility. Subsequently, we utilize these auxiliary points to approximate the Caputo fractional derivative  $\partial_t^\alpha \mathcal{U}(\mathbf{x}, t, \mathcal{I}(\mathbf{x}), \mathcal{J}(t))$  as follows:

$$\begin{aligned} \partial_t^\alpha \mathcal{U}(\mathbf{x}, t, \mathcal{I}(\mathbf{x}), \mathcal{J}(t)) &\approx \delta_{t_{N_t-\sigma}}^\alpha \mathcal{U}(\mathbf{x}, t, \mathcal{I}(\mathbf{x}), \mathcal{J}(t)) \\ &= C_0^{(N_t, \alpha)} \mathcal{U}(\mathbf{x}, t_{N_t}, \mathcal{I}(\mathbf{x}), \mathcal{J}(t_{N_t})) - \sum_{k=0}^{N_t-1} (C_{N_t-k-1}^{(N_t, \alpha)} - C_{N_t-k}^{(N_t, \alpha)}) \mathcal{U}(\mathbf{x}, t_k, \mathcal{I}(\mathbf{x}), \mathcal{J}(t_k)). \end{aligned}$$

During the training stage, we choose  $M_{\tilde{\Gamma}}$  training points located on curved surface  $\tilde{\Gamma} \setminus \Lambda$ , represented as  $\mathcal{P}_{\tilde{\Gamma}} = \{(\mathbf{x}^i, t^i)\}_{i=1}^{M_{\tilde{\Gamma}}}$ , along with  $M_\Lambda$  training points on the interface  $\Lambda$ , denoted by  $\mathcal{P}_\Lambda = \{(\mathbf{x}_\Lambda^i, t_\Lambda^i)\}_{i=1}^{M_\Lambda}$ , and  $M_{T_0}$  training points also situated on curved surface  $\tilde{\Gamma} \setminus \Lambda$ , represented as  $\mathcal{P}_{T_0} = \{x_{T_0}^i\}_{i=1}^{M_{T_0}}$ . Moreover, in association with the training dataset  $\mathcal{P}_{\tilde{\Gamma}}$ , we generate  $(N_t + 1) \times M_{\tilde{\Gamma}}$  auxiliary points, designated as  $\mathcal{P}_{\text{aux}} = \bigcup_{i=1}^{M_{\tilde{\Gamma}}} \mathcal{P}_{\text{aux}}^i$ , where  $\mathcal{P}_{\text{aux}}^i = \{(\mathbf{x}^i, t^j)\}_{j=0}^{N_t}$  are the auxiliary points corresponding to  $(\mathbf{x}^i, t^i) \in \mathcal{P}_{\tilde{\Gamma}}$ . By leveraging the PINNs framework, the loss function associated with the PDE in (2.2a), the jump conditions in (2.2b) and (2.2c), and the initial condition in (2.2d), is represented as follows:

$$\begin{aligned} \text{loss}(\theta) &= \frac{1}{M_{\tilde{\Gamma}}} \sum_{(\mathbf{x}^i, t^i) \in \mathcal{P}_{\tilde{\Gamma}}} L_f(\mathbf{x}^i, t^i, \mathcal{P}_{\text{aux}}^i; \theta)^2 + \frac{1}{M_\Lambda} \sum_{(\mathbf{x}_\Lambda^i, t_\Lambda^i) \in \mathcal{P}_\Lambda} L_{\Lambda_D}(\mathbf{x}_\Lambda^i, t_\Lambda^i; \theta)^2 \\ &\quad + \frac{1}{M_\Lambda} \sum_{(\mathbf{x}_\Lambda^i, t_\Lambda^i) \in \mathcal{P}_\Lambda} L_{\Lambda_N}(\mathbf{x}_\Lambda^i, t_\Lambda^i; \theta)^2 + \frac{1}{M_{T_0}} \sum_{x_{T_0}^i \in \mathcal{P}_{T_0}} L_{T_0}(x_{T_0}^i; \theta)^2, \end{aligned} \quad (3.7)$$

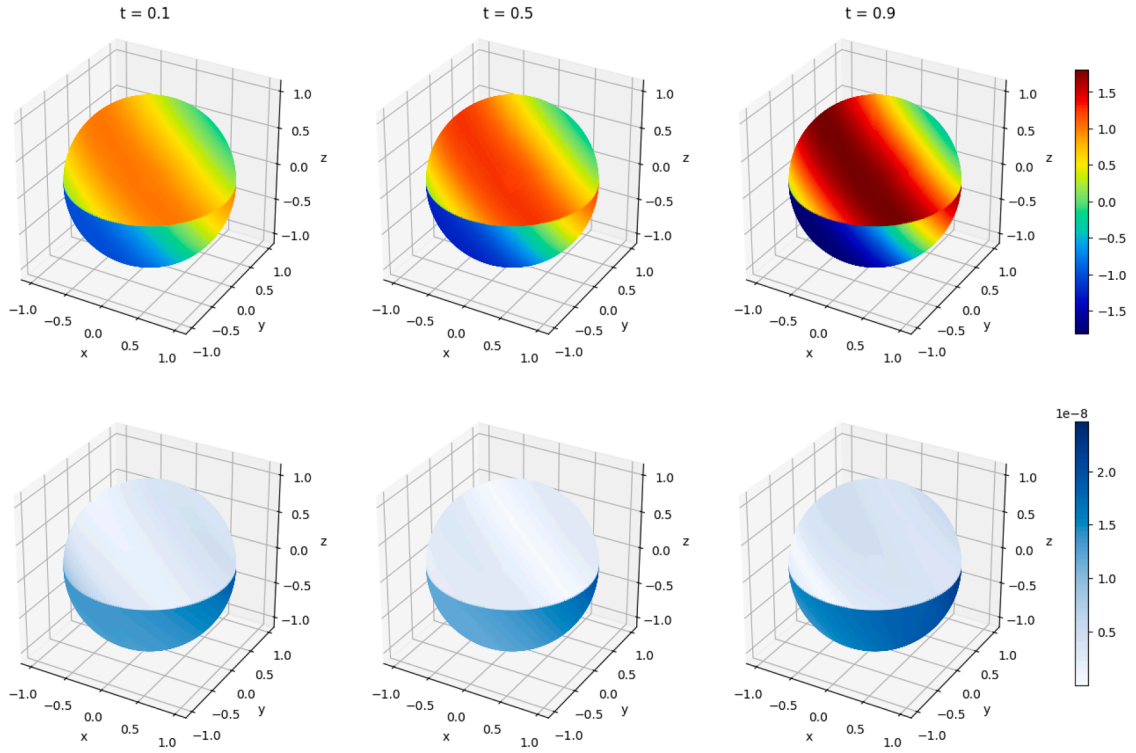
where  $L_f$ ,  $L_{\Lambda_D}$ ,  $L_{\Lambda_N}$ ,  $L_{T_0}$ , represent the residual error of PDE (2.2a), jump conditions (2.2b) and (2.2c), the initial condition (2.2d), respectively. The specific formulations are presented as follows:

$$\begin{aligned} L_f(\mathbf{x}^i, t^i, \mathcal{P}_{\text{aux}}^i; \theta) &= \delta_{t^i}^\alpha \mathcal{U}(\mathbf{x}^i, t^i, \mathcal{I}(\mathbf{x}^i), \mathcal{J}(t^i)) - \nabla_{\tilde{\Gamma}} \cdot \left( \beta \nabla_{\tilde{\Gamma}} \mathcal{U}(\mathbf{x}^i, t^i, \mathcal{I}(\mathbf{x}^i), \mathcal{J}(t^i)) \right) + \kappa \mathcal{U}(\mathbf{x}^i, t^i, \mathcal{I}(\mathbf{x}^i), \mathcal{J}(t^i)) - f(\mathbf{x}^i, t^i), \\ L_{\Lambda_D}(\mathbf{x}_\Lambda^i, t_\Lambda^i; \theta) &= \mathcal{U}(\mathbf{x}_\Lambda^i, t_\Lambda^i, 1, \mathcal{J}(t_\Lambda^i)) - \mathcal{U}(\mathbf{x}_\Lambda^i, t_\Lambda^i, -1, \mathcal{J}(t_\Lambda^i)) - \lambda(\mathbf{x}_\Lambda^i, t_\Lambda^i), \\ L_{\Lambda_N}(\mathbf{x}_\Lambda^i, t_\Lambda^i; \theta) &= \left( \beta^+(\mathbf{x}_\Lambda^i) \nabla_{\tilde{\Gamma}} \mathcal{U}(\mathbf{x}_\Lambda^i, t_\Lambda^i, 1, \mathcal{J}(t_\Lambda^i)) - \beta^-(\mathbf{x}_\Lambda^i) \nabla_{\tilde{\Gamma}} \mathcal{U}(\mathbf{x}_\Lambda^i, t_\Lambda^i, -1, \mathcal{J}(t_\Lambda^i)) \right) \cdot \mathbf{n}_\Lambda - \rho(\mathbf{x}_\Lambda^i, t_\Lambda^i), \\ L_{T_0}(x_{T_0}^i; \theta) &= \mathcal{U}(x_{T_0}^i, 0, \mathcal{I}(x_{T_0}^i), \mathcal{J}(0)) - u_0(x_{T_0}^i). \end{aligned}$$

The aforementioned surface differential terms can be computed using (2.6). During the training process, all constituent loss components are assigned a uniform weight of 1. In the remainder of this paper, we will utilize the ResNet architecture [9] as the primary network framework. An illustration of this architecture is provided in Fig. 2. This choice is motivated by its ability to address the vanishing gradient problem associated with network depth, thereby allowing the network to sustain its performance even as the number of layers increases.

## 4. Numerical experiments

To demonstrate the accuracy, robustness, and effectiveness of the proposed SDC-PINN, numerical tests are conducted in this section. Unless mentioned otherwise, we assign the coefficient  $\kappa$  a value of 1 in all numerical experiments. For the default network configuration, a ResNet architecture featuring  $N_f = 3$  residual blocks is employed. Each block is configured with two hidden layers, and each layer contains  $N_u = 15$  units. We select the hyperbolic tangent function, defined as  $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ , for the activation function. Additionally, the geodesic acceleration Levenberg-Marquardt (gdlm) method is applied for optimizing the network. The



**Fig. 3.** Example 1: the numerical results for the case without initial singularity at times  $t = 0.1, 0.5$ , and  $0.9$ . Top: the network solutions; Bottom: the error distributions.

training is stopped upon reaching the maximum training epoch (2000) or upon satisfying the convergence criteria outlined in [25]. Moreover, the hyperparameter  $N_l$  is set to 128, while  $N_\varepsilon$  is assigned a value of 1 and  $r$  is determined as  $2/\alpha$ . An initial point set is generated using either the surface's level set function or the DistMesh package [17]. Random subsets are then sampled from this set for model training. The testing dataset consists of approximately  $\mathcal{O}(10^4)$  points.

**Example 1.** In the first case, we consider the surface  $\tilde{\Gamma}$  to be a unit sphere. The interface is defined by  $z = 0$ ,  $x^2 + y^2 = 1$ . The sub-surfaces  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$  are defined as the upper half and lower half of  $\tilde{\Gamma}$ , respectively. The chosen analytic solution in the region  $\tilde{\Gamma}^+$  is  $(r^2 + 1)\cos(x + y + z)$ , whereas in the region  $\tilde{\Gamma}^-$ , it is expressed as  $(r^2 + 1)\sin(x + y + z)$ , and the parameter  $\alpha$  is assigned a value of 0.25. In the region  $\tilde{\Gamma}^+$ , we define the diffusion coefficient  $\beta$  as  $\sin(x + y + z) + 2$ , whereas in the region  $\tilde{\Gamma}^-$ , it is represented by  $\cos(x + y + z) + 2$ .

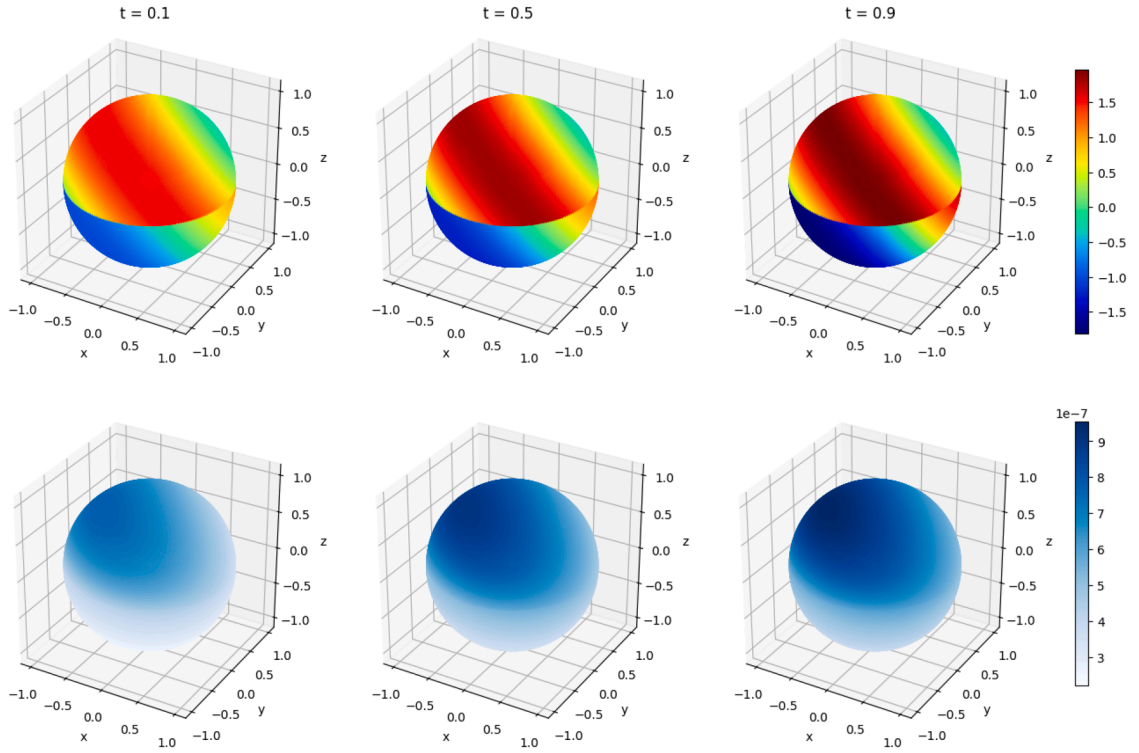
In this scenario, we first assess the ability of the SDC-PINN to tackle problem (2.2) in the absence of an initial singularity. The quantities of training points are selected as follows:  $M_\Gamma = 4500$ ,  $M_\Lambda = 1180$ , and  $M_{T_0} = 1000$ . In addition, we set  $N_l = 1$  and  $N_u = 20$  for this particular case. Given that the solution does not possess an initial singularity, we utilize the uniform L2-1 $_\sigma$  scheme to approximate the Caputo fractional derivative  $\partial_t^\alpha \mathcal{U}$ . The calculated relative  $L^2$ -error in this scenario attains a value of  $8.4269 \times 10^{-9}$ , whereas the computed  $L^\infty$ -error is  $3.0628 \times 10^{-8}$ . Fig. 3 presents the numerical solutions along with the error distributions at times  $t = 0.1, 0.5, 0.9$ . This example successfully demonstrates the ability of the SDC-PINN to achieve precise numerical solutions for time-fractional diffusion problems with discontinuous coefficients on curved surfaces.

To further assess the effectiveness of the SDC-PINN in approximating solutions that exhibit initial singularities, we perform a comparable experiment. The surface, interface, and coefficients utilized are consistent with those in the previous test. The analytic solution, featuring an initial singularity, is characterized by  $(r^\alpha + 1)\cos(x + y + z)$  in the region  $\tilde{\Gamma}^+$ , while in the region  $\tilde{\Gamma}^-$ , it is taken as  $(r^2 + 1)\sin(x + y + z)$ .

In this case, the nonuniform L2-1 $_\sigma$  scheme is utilized to estimate the Caputo fractional derivative associated with the output of the neural network. The solution on the upper hemispherical surface evidently exhibits an initial singularity, whereas the solution on the lower hemispherical surface maintains its non-singularity at the initial time. The numerical results are illustrated in Fig. 4. For this case, the relative error in  $L^2$ -norm and the  $L^\infty$ -error reach values of  $4.7864 \times 10^{-7}$  and  $3.1248 \times 10^{-6}$ , respectively, indicating that the SDC-PINN is skilled in resolving such problems that involve initial singularities.

**Example 2.** To further test the capability of the SDC-PINN, we consider problem (2.2) involving an initial singularity on a torus expressed by

$$(\sqrt{x^2 + y^2} - 1)^2 + z^2 - 1/16 = 0,$$



**Fig. 4.** Example 1: the numerical results for the case with initial singularity at times  $t = 0.1, 0.5$ , and  $0.9$ . Top: the network solutions; Bottom: the error distributions.

**Table 1**  
The accuracy in Example 2.

| $N_l$ | $N_u$ | Relative $L^2$ -error | $L^\infty$ -error |
|-------|-------|-----------------------|-------------------|
| 1     | 8     | 1.71E-06              | 7.72E-05          |
| 1     | 10    | 8.72E-07              | 3.29E-06          |
| 1     | 15    | 8.72E-07              | 2.93E-06          |
| 2     | 8     | 8.87E-07              | 3.35E-06          |
| 2     | 10    | 8.70E-07              | 3.09E-06          |
| 2     | 15    | 8.55E-07              | 2.89E-06          |
| 3     | 8     | 8.80E-07              | 2.93E-06          |
| 3     | 10    | 8.59E-07              | 2.90E-06          |
| 3     | 15    | 8.51E-07              | 2.95E-06          |

We establish the interface  $\Lambda$  in the following manner:

$$x = 0, (\sqrt{y^2} - 1)^2 + z^2 - 1/16 = 0.$$

The sub-surface characterized by  $\tilde{\Gamma}^+ = \{x|x > 0\} \cap \tilde{\Gamma}$ , along with  $\tilde{\Gamma}^- = \{x|x < 0\} \cap \tilde{\Gamma}$ , is under consideration. Below is the presentation of the provided analytic solution:

$$u = \begin{cases} (1 + t^\alpha) \sin x e^{\cos(y+z)}, & \text{in } \tilde{\Gamma}^+, \\ (1 + t^\alpha) \cos x e^{\sin(y+z)}, & \text{in } \tilde{\Gamma}^-. \end{cases}$$

For this test case, the time-fractional order  $\alpha$  is set to 0.25. On the sub-surface  $\tilde{\Gamma}^+$ , we characterize the diffusion coefficient  $\beta$  as  $\sin(x + y + z) + 2$ , while on the sub-surface  $\tilde{\Gamma}^-$ , it is expressed as  $\cos(x + y + z) + 2$ .

For this case, the influence of the number of trainable parameters is investigated. We alter the quantity of residual blocks from 1 to 3, as well as the number of units in each hidden layer, which ranges from 8 to 15. For this study, we selected 3600 training points from  $\tilde{\Gamma}$ , 450 training points from  $\Lambda$ , and an additional 800 training points at  $t = 0$ . Table 1 illustrates the relative  $L^2$ -error associated with different configurations of residual blocks and neurons in hidden layers. As expected, an increase in the quantity of residual blocks and units enhances the accuracy of the numerical solution. Remarkably, a neural network consisting of just one residual block with eight neurons in each layer can achieve an accuracy level of  $\mathcal{O}(10^{-6})$  in the relative  $L^2$ -norm error. Moreover, the lowest relative  $L^2$ -error and  $L^\infty$ -error achieved by the SDC-PINN are approximately  $\mathcal{O}(10^{-7})$  and  $\mathcal{O}(10^{-6})$ , respectively, revealing that our approach

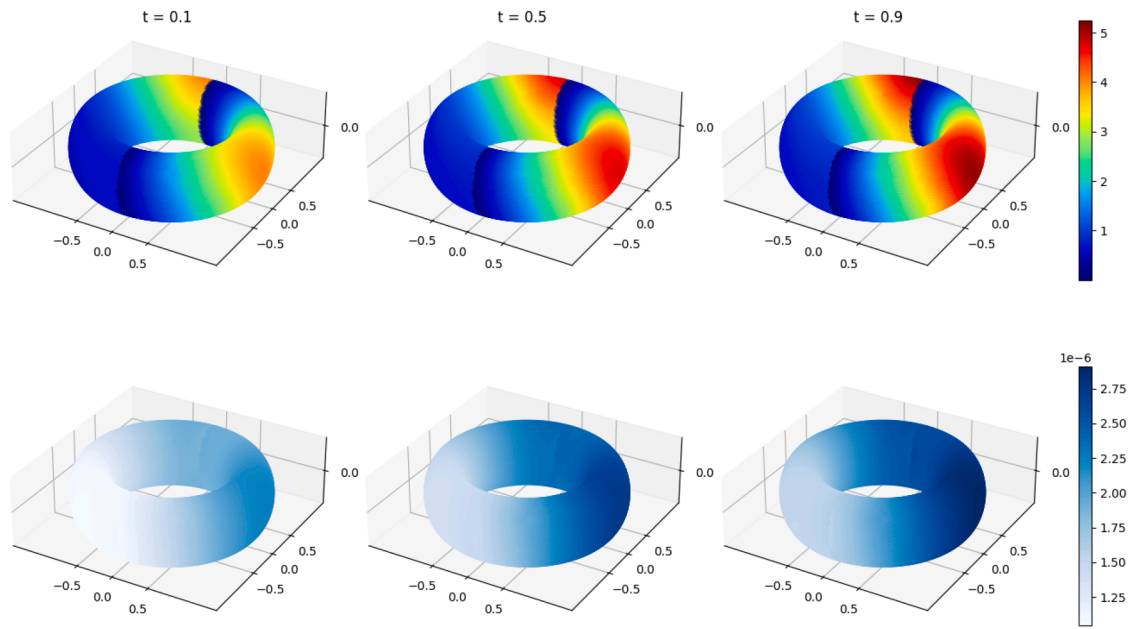


Fig. 5. Example 2: the network solutions (Top); the error distributions (Bottom). Here, the network hyperparameters are set as  $N_l = 3$ ,  $N_u = 15$ .

Table 2

Key configurations of all models for the comparative study in Examples 2 and 3.

| Method               | Network Architecture                              | No. of Params            | Scheme                                               | Key Features                                                                                                                          |
|----------------------|---------------------------------------------------|--------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| SDC-PINN             | $N_l = 3$ , $N_u = 15$                            | $1546 + 16 \times N_\xi$ | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | $N_\xi = 1$ for Example 2, $N_\xi = 3$ for Example 3.                                                                                 |
| DCSNN                | $N_l = 3$ , $N_u = 15$                            | 1546                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | —                                                                                                                                     |
| Decoupling           | $N_l = 3$ , $N_u = 15$                            | 1787                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | Uses a shallow network (40 units) for the first stage.<br>Cusp-enforcing level set function: $ x $ for Example 2, $ z $ for Example 3 |
| Coupling             | $N_l = 3$ , $N_u = 15$                            | 1547                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | —                                                                                                                                     |
| PWDNN                | $2 \times \text{ResNet}$ , $N_l = 3$ , $N_u = 10$ | 1442                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | —                                                                                                                                     |
| fPINNs               | $N_l = 3$ , $N_u = 15$                            | 1531                     | Uniform L1 ( $r = 1$ )                               | —                                                                                                                                     |
| fPINNs <sup>+</sup>  | $N_l = 3$ , $N_u = 15$                            | 1531                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | —                                                                                                                                     |
| IFPINNs              | $N_l = 3$ , $N_u = 15$                            | 1576                     | Nonuniform L1 ( $r = \frac{2-\alpha}{\alpha}$ )      | Initial transformation employs a cubic polynomial.                                                                                    |
| IFPINNs <sup>+</sup> | $N_l = 3$ , $N_u = 15$                            | 1576                     | Nonuniform L2-1 $_\sigma$ ( $r = \frac{2}{\sigma}$ ) | Initial transformation employs a cubic polynomial.                                                                                    |

maintains exceptional performance when tackling problem (2.2) with initial singularity. Fig. 5 displays the outcomes attained by the SDC-PINN with  $N_u = 15$  neurons per hidden layer and  $N_l = 3$  residual blocks at times  $t = 0.1, 0.5$ , and  $0.9$ , respectively.

Next, we conduct a comprehensive comparative analysis to evaluate the performance of our proposed SDC-PINN against several benchmark neural network methods. The competing models include discontinuity capturing shallow neural network (DCSNN) [10], the decoupling DR-PINN (Decoupling) and coupling DR-PINN (Coupling) [30,31], the piecewise deep neural network (PWDNN) [8]. To provide a fair basis for comparison, these models are all configured to employ the nonuniform L2-1 $_\sigma$  scheme and a ResNet architecture. For the decoupling DR-PINN, we utilize a shallow neural network consisting of one hidden layer and 40 units in the first stage to approximate the jump condition (2.2b). The cusp-enforcing level set function is defined as  $|x|$ . Furthermore, we include the fPINNs [16] and the IFPINNs [19] in our comparison. In their original formulations, fPINNs and IFPINNs utilize the uniform L1 and nonuniform L1 schemes, respectively, for approximating the Caputo fractional derivative (2.5). Therefore, to maintain methodological consistency across all models, enhanced versions, denoted as fPINNs<sup>+</sup> and IFPINNs<sup>+</sup>, are also implemented. These variants are modified to utilize the same nonuniform L2-1 $_\sigma$  scheme. Additionally, both IFPINNs and IFPINNs<sup>+</sup> utilize a cubic polynomial for the initial transformation in the network, as defined by  $s_1 = \tanh(\sum_{i=1}^3 W_{0,i}(s_0)^i + b_0)$ . The key configurations of all tested models are summarized in Table 2.

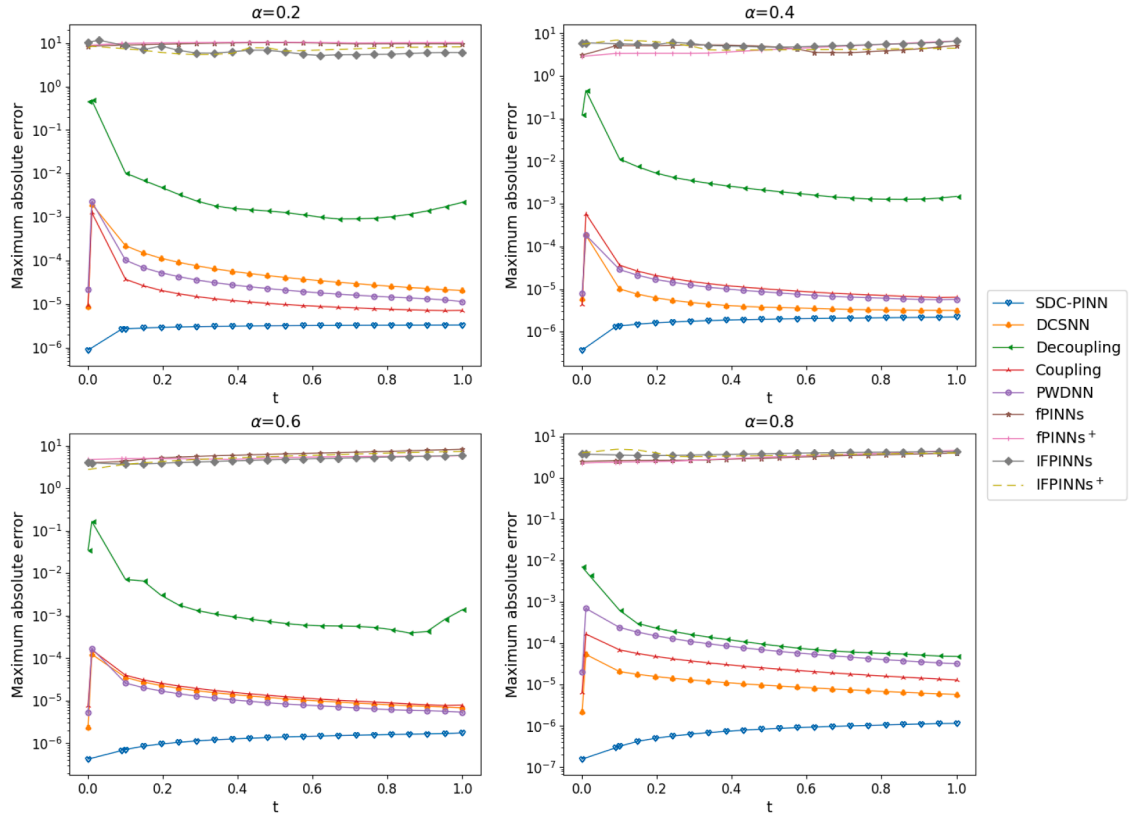
Table 3 presents the comparative performance of the models. Fig. 6 plots the  $L^\infty$ -errors progression over time. Fig. 7 depicts the training loss and the validation loss for  $f$  over epochs.

It can be observed that the fPINNs, fPINNs<sup>+</sup>, IFPINNs, and IFPINNs<sup>+</sup> exhibit the poorest performance. The loss curves in Fig. 7 for these methods stagnate at high values, indicating significant difficulty in minimizing the loss function. This behavior is attributed to the inability of these architectures to effectively model the spatial discontinuity inherent in the problem (2.2).

**Table 3**

The accuracy of the comparative study in Example 2.

| Method               | $\alpha = 0.2$        |                   | $\alpha = 0.4$        |                   | $\alpha = 0.6$        |                   | $\alpha = 0.8$        |                   |
|----------------------|-----------------------|-------------------|-----------------------|-------------------|-----------------------|-------------------|-----------------------|-------------------|
|                      | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error |
| SDC-PINN             | 9.70E-07              | 3.33E-06          | 6.36E-07              | 2.21E-06          | 4.72E-07              | 1.73E-06          | 3.07E-07              | 1.16E-06          |
| DCSNN                | 1.22E-03              | 1.79E-01          | 7.94E-05              | 2.45E-02          | 1.82E-05              | 2.77E-03          | 6.85E-06              | 2.66E-04          |
| PWDNN                | 1.20E-03              | 2.82E-01          | 1.34E-04              | 3.07E-02          | 2.77E-05              | 3.72E-03          | 4.02E-05              | 1.56E-03          |
| Decoupling           | 1.16E-02              | 5.46E-01          | 6.73E-03              | 3.63E-01          | 2.55E-03              | 1.61E-01          | 1.16E-04              | 4.22E-03          |
| Coupling             | 8.40E-04              | 2.36E-01          | 1.04E-04              | 2.66E-02          | 2.39E-05              | 3.10E-03          | 1.63E-05              | 5.16E-04          |
| fPINNs               | 3.43E-01              | 1.04E+01          | 4.18E-01              | 6.15E+00          | 2.74E-01              | 5.67E+00          | 2.45E-01              | 4.65E+00          |
| fPINNs <sup>+</sup>  | 3.76E-01              | 1.01E+01          | 5.11E-01              | 5.34E+00          | 3.41E-01              | 8.14E+00          | 2.55E-01              | 3.93E+00          |
| IFPINNs              | 3.83E-01              | 7.89E+00          | 1.81E-01              | 5.60E+00          | 6.65E-01              | 7.32E+00          | 2.29E-01              | 4.01E+00          |
| IFPINNs <sup>+</sup> | 3.67E-01              | 6.93E+00          | 1.77E-01              | 6.00E+00          | 2.74E-01              | 5.79E+00          | 2.34E-01              | 4.29E+00          |

**Fig. 6.** Example 2: the maximum absolute errors over time for different methods.

In contrast, the DCSNN, DR-PINNs, and PWDNN models are capable of effectively minimizing the training loss. However, these methods struggle to handle the initial singularity of the solution. As shown in Fig. 7, they exhibit high validation losses for  $f$ . Additionally, Fig. 6 reveals their tendency to overfit the solution in the latter part of the time interval and present large approximation errors in the vicinity of the initial time.

The proposed SDC-PINN framework successfully addresses these challenges by integrating both discontinuity- and singularity-capturing features. As a result, it achieves a significant improvement in accuracy when compared to the other benchmarked methods.

**Example 3.** For this test case, the computation domain of problem (2.2) is considered to be the surface of a cube, denoted by

$$\phi = (x^4 + y^4 + z^4) - 0.5(x^2 + y^2 + z^2).$$

We select the interface as  $\Lambda = \tilde{\Gamma} \cap \{(x, y, z) | z = 0\}$ . The sub-surfaces  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$  are defined as the parts of  $\tilde{\Gamma}$  where  $z > 0$  and  $z < 0$ , respectively. The chosen exact solution is provided by

$$u = \begin{cases} (\sin t^\alpha + t) \cos(x + y + z), & \text{in } \tilde{\Gamma}^+, \\ (\cos t^\alpha + t) \sin(x + y + z), & \text{in } \tilde{\Gamma}^-. \end{cases}$$

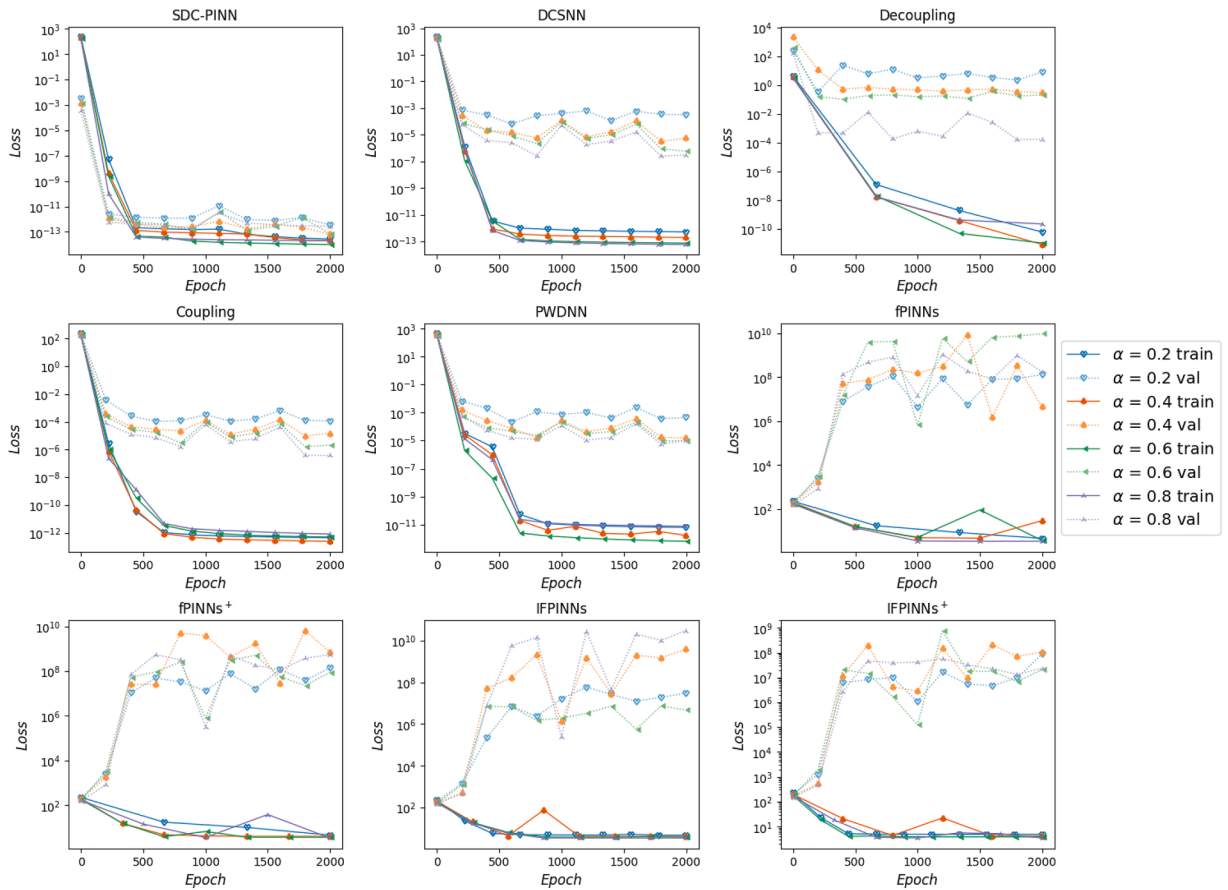
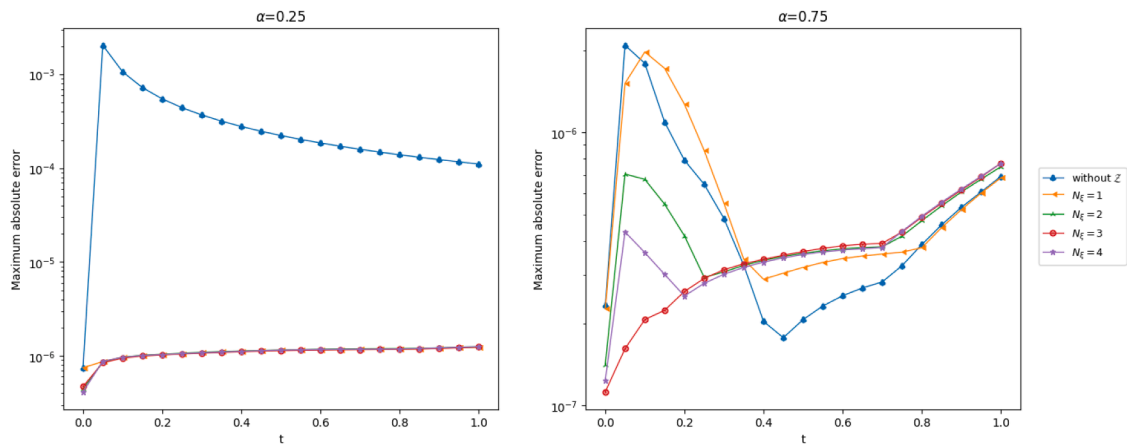
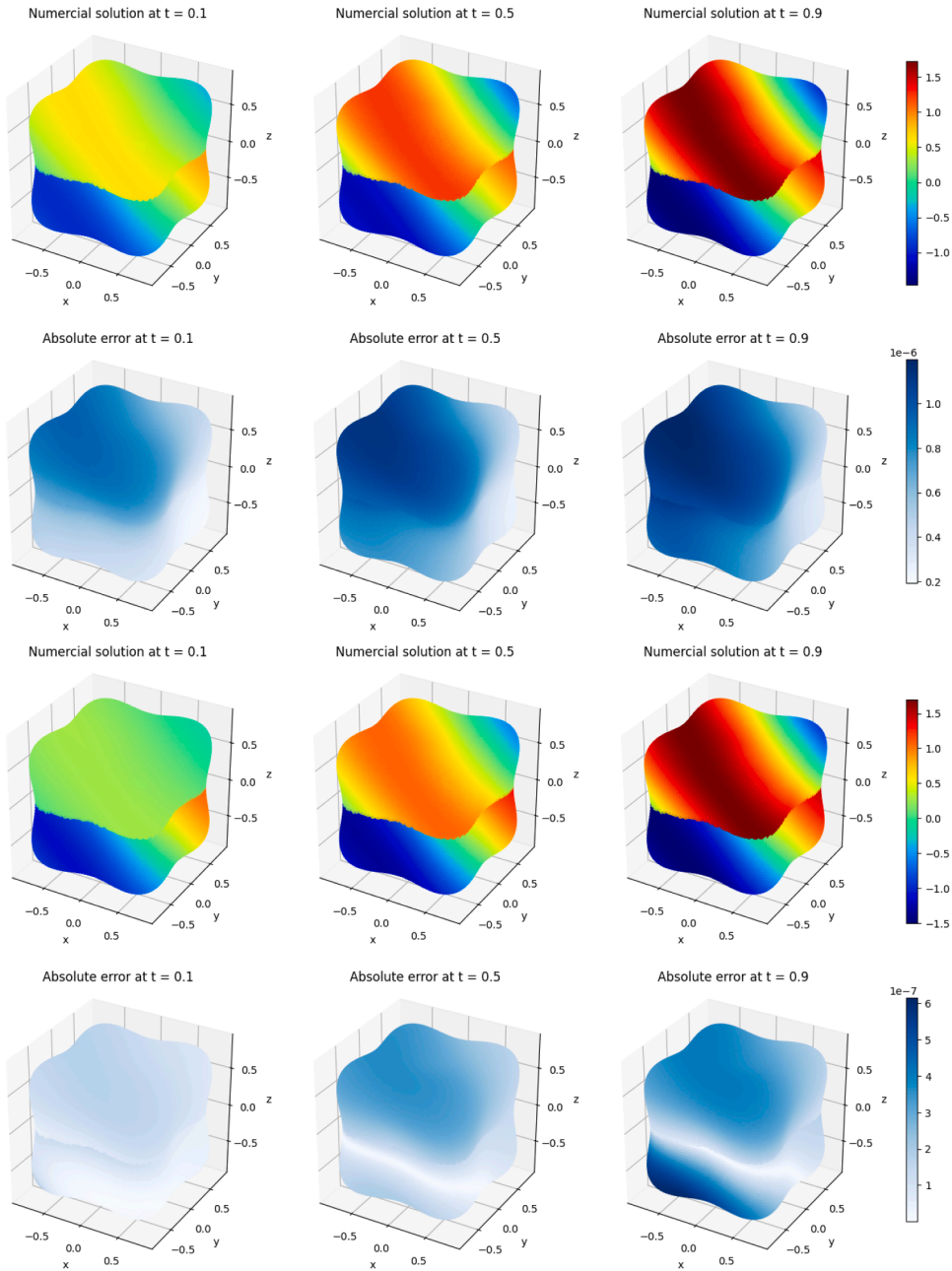
Fig. 7. Example 2: the training loss and validation loss for  $f$ .

Fig. 8. Example 3: the maximum absolute errors over time.

In the region  $\tilde{\Gamma}^+$ , the diffusion coefficient  $\beta$  is expressed as  $\ln(2 + x^2 + y^2 + z^2)$ , while in the region  $\tilde{\Gamma}^-$ , it is given by  $e^{x+y+z}$ .

In contrast to the previous examples, the solution  $u$  presented here exhibits a more complex initial singularity, as illustrated by the Taylor expansion

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, \quad \cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots.$$



**Fig. 9.** Example 3: the network solutions and error distributions generated by SDC-PINN with  $N_\xi$  fixed at 3. Top two rows:  $\alpha = 0.25$ ; Bottom two rows:  $\alpha = 0.75$ .

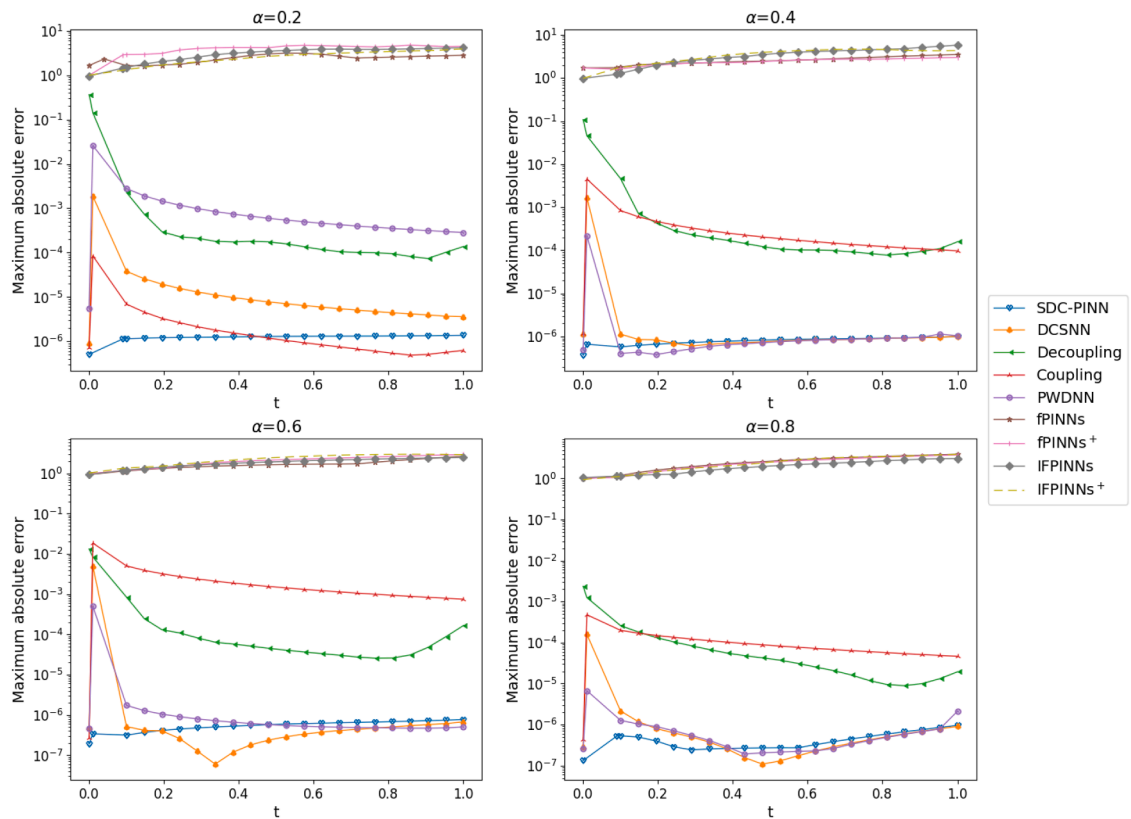
**Table 4**  
The accuracy in Example 3.

| $N_\xi$ | $\alpha = 0.25$       |                   | $\alpha = 0.75$       |                   |
|---------|-----------------------|-------------------|-----------------------|-------------------|
|         | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error |
| 0       | 1.38E-02              | 1.94E+00          | 1.23E-04              | 2.20E-02          |
| 1       | 1.06E-06              | 1.20E-04          | 2.46E-07              | 1.92E-06          |
| 2       | 9.32E-07              | 5.88E-05          | 2.54E-07              | 1.21E-06          |
| 3       | 8.68E-07              | 3.80E-05          | 2.58E-07              | 7.54E-07          |
| 4       | 8.58E-07              | 2.59E-05          | 2.53E-07              | 1.23E-06          |

**Table 5**

The accuracy of the comparative study in Example 3.

| Method               | $\alpha = 0.2$        |                   | $\alpha = 0.4$        |                   | $\alpha = 0.6$        |                   | $\alpha = 0.8$        |                   |
|----------------------|-----------------------|-------------------|-----------------------|-------------------|-----------------------|-------------------|-----------------------|-------------------|
|                      | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error |
| SDC-PINN             | 1.01E-06              | 1.09E-05          | 5.18E-07              | 1.18E-06          | 3.13E-07              | 7.69E-07          | 2.29E-07              | 9.46E-07          |
| DCSNN                | 2.66E-02              | 2.54E+00          | 1.20E-02              | 1.08E+00          | 2.77E-03              | 3.63E-01          | 1.65E-04              | 2.14E-02          |
| PWDNN                | 4.61E-02              | 1.74E+00          | 1.33E-03              | 1.26E-01          | 3.18E-04              | 3.46E-02          | 3.90E-06              | 6.32E-04          |
| Decoupling           | 5.64E-03              | 1.52E-01          | 1.45E-03              | 4.46E-02          | 1.96E-04              | 8.37E-03          | 5.11E-05              | 1.25E-03          |
| Coupling             | 2.63E-04              | 2.64E-02          | 6.82E-03              | 4.22E-01          | 6.55E-03              | 2.21E-01          | 1.51E-04              | 9.47E-04          |
| fPINNs               | 5.08E-01              | 4.66E+00          | 5.63E-01              | 2.96E+00          | 4.02E-01              | 2.72E+00          | 5.25E-01              | 3.39E+00          |
| fPINNs <sup>+</sup>  | 5.24E-01              | 2.81E+00          | 4.54E-01              | 3.47E+00          | 4.47E-01              | 2.32E+00          | 5.18E-01              | 3.85E+00          |
| IFPINNs              | 5.83E-01              | 3.53E+00          | 6.43E-01              | 4.67E+00          | 5.81E-01              | 2.77E+00          | 5.94E-01              | 3.78E+00          |
| IFPINNs <sup>+</sup> | 6.62E-01              | 4.00E+00          | 7.74E-01              | 4.84E+00          | 4.99E-01              | 2.42E+00          | 4.89E-01              | 3.02E+00          |

**Fig. 10.** Example 3: the maximum absolute errors over time for different methods.

We explore the influence of the hyperparameter  $N_\xi$ , which ranges from 0 to 4. We allocate 4500 training points on  $\tilde{\Gamma}$ , 590 training points on  $\Lambda$ , and 800 training points at time  $t = 0$ . Table 4 and Fig. 8 present the accuracy and the time evolution of the  $L^\infty$ -errors, respectively. Fig. 9 illustrates the network solutions and error distributions obtained by SDC-PINN with  $N_\xi = 3$ . It is highlighted that the neural network lacking the feature  $\mathcal{Z}$  is unable to capture the initial singularity, especially when  $\alpha = 0.25$ , only achieving an accuracy of approximately  $10^{-2}$ . Furthermore, it is observed that setting  $N_\xi = 1$  is adequate for  $\mathcal{U}$  to capture the initial singularity, which leads to an accuracy improvement of four orders of magnitude in the  $L^\infty$  norm compared to  $N_\xi = 0$ . Additionally, the numerical findings reveal that increasing  $N_\xi$  enables a more accurate approximation of the complex initial singularity, consequently leading to more precise numerical solutions. The final relative  $L^2$ -error of the SDC-PINN achieves a magnitude of  $\mathcal{O}(10^{-7})$ , highlighting the ability of the SDC-PINN to tackle problems that exhibit more complex initial singularities.

A comparative study analogous to Example 2 is performed for this case, using the configurations in Table 2. Table 5 presents the numerical results. Fig. 10 plots the temporal evolution of the  $L^\infty$ -error. Fig. 11 depicts the training loss and the validation loss for  $f$  dynamics over epochs.

Consistent with the findings in Example 2, the fPINNs, fPINNs<sup>+</sup>, IFPINNs, and IFPINNs<sup>+</sup> perform poorly due to their inability to resolve the spatial discontinuity. Furthermore, the performance of the DCSNN, PWDNN, and DR-PINNs also deteriorates in this more

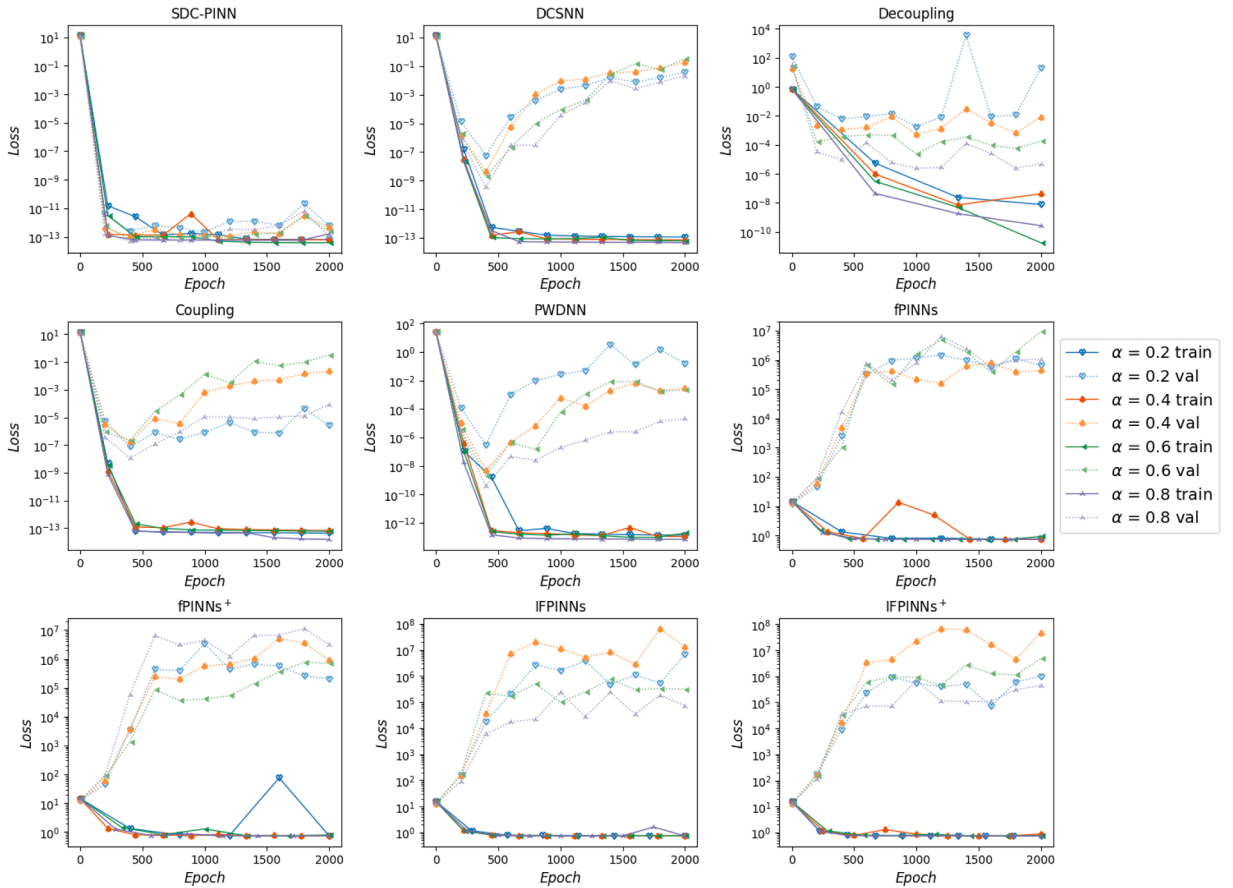


Fig. 11. Example 3: the training loss and validation loss for  $f$  for different methods.

challenging case, exhibiting higher initial errors and unstable validation losses. This is attributed to the more complex initial singularity of the solution. In contrast, the proposed SDC-PINN leverages its singularity-capturing feature to effectively handle this increased complexity. This allows it to achieve robust convergence and the lowest errors among all benchmarked models, demonstrating its superior capability.

**Example 4.** In the fourth example, the problem (2.2) is considered on the surface defined by

$$\begin{aligned} \phi = & (4x^2 - 1)^2 + (4y^2 - 1)^2 + (4z^2 - 1)^2 + \\ & 16(x^2 + y^2 - 1)^2 + 16(x^2 + z^2 - 1)^2 + 16(y^2 + z^2 - 1)^2 - 16. \end{aligned}$$

The expression for the interface  $\Lambda$  is:

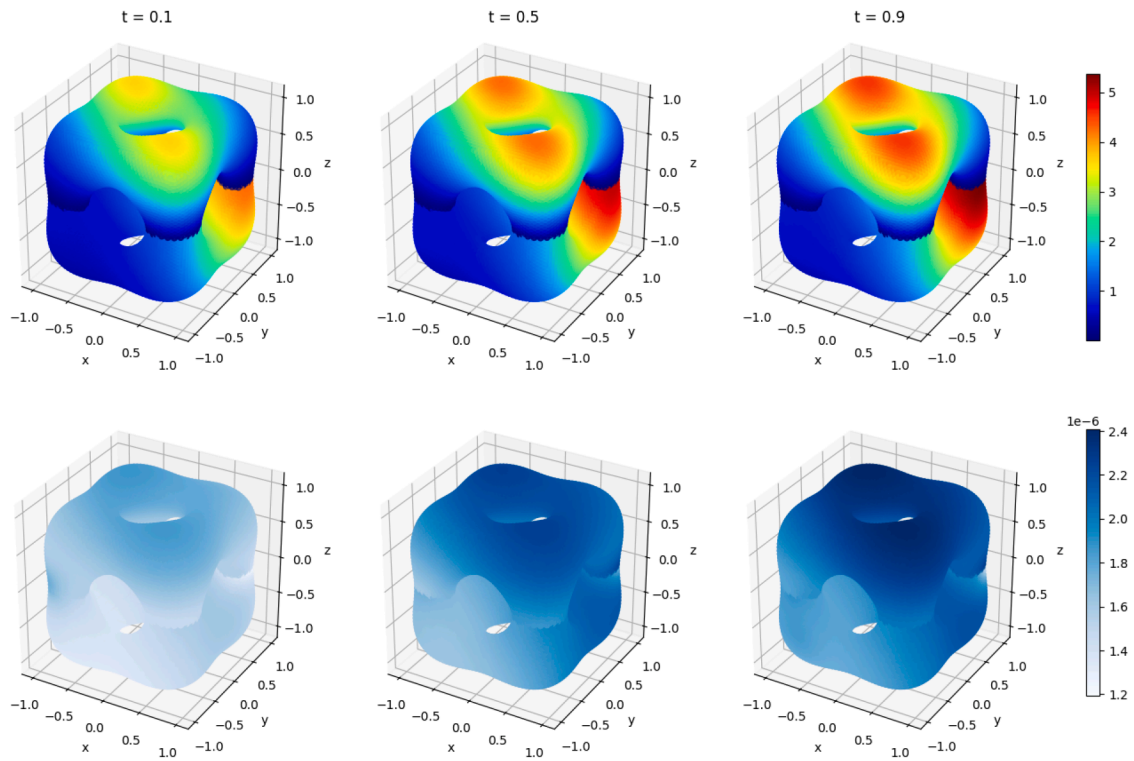
$$(4x^2 - 1)^2 + (4y^2 - 1)^2 + 16(x^2 + y^2 - 1)^2 + 16(x^2 - 1)^2 + 16(y^2 - 1)^2 - 15, \quad z = 0.$$

Subsequently,  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$  are taken as the upper half and lower half of  $\tilde{\Gamma}$ . The time-fractional order is set as  $\alpha = 0.25$ . The analytic solution is selected as  $(t^\alpha + 1) \sin z e^{\cos(x+y)}$  in the region  $\tilde{\Gamma}^+$ , while in the region  $\tilde{\Gamma}^-$ , it is taken as  $(t^\alpha + 1) \cos z e^{\sin(x+y)}$ . In the region  $\tilde{\Gamma}^+$ , the diffusion coefficient  $\beta$  is defined as  $\sin(x + y + z) + 2$ , whereas in the region  $\tilde{\Gamma}^-$ , it is represented by  $\cos(x + y + z) + 2$ .

Numerical outcomes for this specific scenario are presented in Table 6 and illustrated in Fig. 12. Table 6 showcases the resulting  $L^\infty$ -error alongside the relative  $L^2$ -error, which varies with the number of training points. It is noted that the  $L^\infty$ -error decreases as  $M_{\tilde{\Gamma}}$  and  $M_{T_0}$  rise, but shows little sensitivity to changes in  $M_\Lambda$ . This trend indicates that augmenting the number of training points around  $t = 0$  allows the neural network  $\mathcal{U}$  to efficiently capture the initial conditions. Consequently,  $\xi$  adapts more swiftly to the initial singularity of the solution, leading to enhanced solution precision. It is noted that the smallest relative error in the  $L^2$  norm achieved by the SDC-PINN is around  $10^{-7}$ , indicating that the SDC-PINN is proficient in addressing the problem outlined in (2.2) involving complex surface geometry.

**Example 5.** In this scenario, problem (2.2) is considered on a peanut surface expressed by

$$\phi = 2((2x + 1)^2 + 4y^2 + 4z^2)((2x - 1)^2 + 4y^2 + 4z^2) - 3.$$



**Fig. 12.** Example 4: the network solutions (Top); the error distributions (Bottom) at times  $t = 0.1, 0.5$ , and  $0.9$ , respectively. Here, the numbers of training points are set as follows:  $M_{\tilde{\Gamma}} = 3600$ ,  $M_{\Lambda} = 354$ ,  $M_{T_0} = 1000$ .

**Table 6**  
The numerical results in Example 4.

| $M_{\tilde{\Gamma}}$ | $M_{\Lambda}$ | $M_{T_0}$ | $L^{\infty}$ -error | Relative $L^2$ -error |
|----------------------|---------------|-----------|---------------------|-----------------------|
| 1200                 | 354           | 1000      | 8.65E-07            | 2.51E-05              |
| 2400                 | 354           | 1000      | 8.68E-07            | 4.84E-06              |
| 3600                 | 118           | 1000      | 8.52E-07            | 3.21E-06              |
| 3600                 | 236           | 1000      | 8.58E-07            | 3.07E-06              |
| 3600                 | 354           | 100       | 8.77E-07            | 7.09E-06              |
| 3600                 | 354           | 600       | 8.53E-07            | 2.91E-06              |
| 3600                 | 354           | 1000      | 8.53E-07            | 2.49E-06              |

The interface  $\Lambda$  is characterized as the curve at which  $\tilde{\Gamma}$  meets the plane at  $x = 0$ . Let the upper half of the peanut surface be denoted as  $\tilde{\Gamma}^+$ , and the lower half as  $\tilde{\Gamma}^-$ . We choose the analytic solution as

$$u = \begin{cases} (t^{\alpha} + 1) \sin x e^{\cos(y+z)}, & \text{in } \tilde{\Gamma}^+, \\ (t^{\alpha} + 1) \cos x e^{\sin(y+z)}, & \text{in } \tilde{\Gamma}^-, \end{cases}$$

On the sub-surface  $\tilde{\Gamma}^+$ , the diffusion coefficient  $\beta = \sin(x + y + z) + 2$  is used, whereas on  $\tilde{\Gamma}^-$ , it is defined by  $\beta = \cos(x + y + z) + 2$ .

For this test case, we evaluate the performance of different finite difference techniques in the temporal direction, namely the uniform L1 scheme, the uniform L2-1 $_{\sigma}$  scheme, the nonuniform L1 scheme, and the nonuniform L2-1 $_{\sigma}$  scheme. We designate the hyperparameters as follows:  $M_{\tilde{\Gamma}} = 4500$ ,  $M_{\Lambda} = 500$ , and  $M_{\partial\tilde{\Gamma}} = 800$ . The relative  $L^2$ -error and the  $L^{\infty}$  error for  $\mathcal{U}$  are shown in Table 7, while Fig. 13 demonstrates the  $L^{\infty}$ -errors over time. For  $\alpha = 0.25$ , it is observed that schemes utilizing graded meshes exhibit superior accuracy compared to those employing uniform meshes when addressing problem (2.2), which features an initial singularity. This enhanced precision is due to the graded mesh's capability of offering a more accurate approximation of  $\partial_t^{\alpha} \mathcal{U}$  in the vicinity of  $t = 0$ . The numerical findings indicate that the nonuniform L2-1 $_{\sigma}$  scheme outperforms the other schemes in approximating  $\partial_t^{\alpha} \mathcal{U}$  in the presence of an initial singularity. In contrast, when  $\alpha = 0.75$ , the initial singularity in the solution is considerably milder than at  $\alpha = 0.25$ , leading to diminished advantages of using a graded mesh. Lastly, Fig. 14 illustrates the network solutions and error distributions obtained by our SDC-PINN utilizing the nonuniform L2-1 $_{\sigma}$  scheme. One can clearly see that the smallest  $L^{\infty}$  error is approximately  $10^{-6}$ , indicating the effectiveness of the proposed SDC-PINN and the nonuniform L2-1 $_{\sigma}$  scheme.

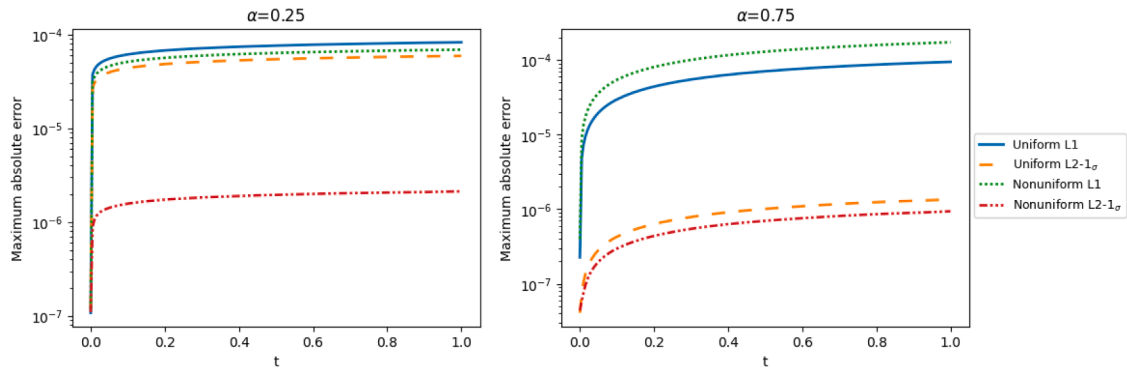


Fig. 13. Example 5: the maximum absolute errors over time.

**Table 7**  
The accuracy in Example 5.

| Finite difference technique                  | $\alpha = 0.25$       |                   | $\alpha = 0.75$       |                   |
|----------------------------------------------|-----------------------|-------------------|-----------------------|-------------------|
|                                              | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error |
| Uniform L1 ( $r = 1$ )                       | 3.74E-05              | 8.25E-05          | 3.95E-05              | 9.43E-05          |
| Uniform L2- $1_\sigma$ ( $r = 1$ )           | 2.68E-05              | 5.91E-05          | 5.65E-07              | 1.35E-06          |
| Nonuniform L1 ( $r = 2 - \alpha/\alpha$ )    | 3.12E-05              | 6.89E-05          | 7.23E-05              | 1.73E-04          |
| Nonuniform L2- $1_\sigma$ ( $r = 2/\alpha$ ) | 9.59E-07              | 2.12E-06          | 3.91E-07              | 9.38E-07          |

**Table 8**  
The numerical results in Example 6.

| $N_t$ | $\alpha = 0.25$       |                   | $\alpha = 0.75$       |                   |
|-------|-----------------------|-------------------|-----------------------|-------------------|
|       | Relative $L^2$ -error | $L^\infty$ -error | Relative $L^2$ -error | $L^\infty$ -error |
| 32    | 4.20E-06              | 3.62E-06          | 1.16E-06              | 1.39E-07          |
| 64    | 3.58E-06              | 3.28E-06          | 8.92E-07              | 1.09E-07          |
| 96    | 3.30E-06              | 3.35E-06          | 8.50E-07              | 1.04E-07          |
| 128   | 2.95E-06              | 2.88E-06          | 8.06E-07              | 9.52E-08          |
| 160   | 3.09E-06              | 3.20E-06          | 8.03E-07              | 1.01E-07          |

**Example 6.** Next, we consider  $\tilde{\Gamma}$  to be a Genes-2 torus defined by

$$\phi = [(x+1)x^2(x-1) + y^2]^2 + z^2 - 0.01.$$

The expression of  $\Lambda$  is given by

$$y^4 + z^2 = 0.01, \quad x = 0.$$

We refer to the upper half and the lower half of Genes-2 torus as  $\tilde{\Gamma}^+$  and  $\tilde{\Gamma}^-$ , respectively. The selected analytic solution has the form of

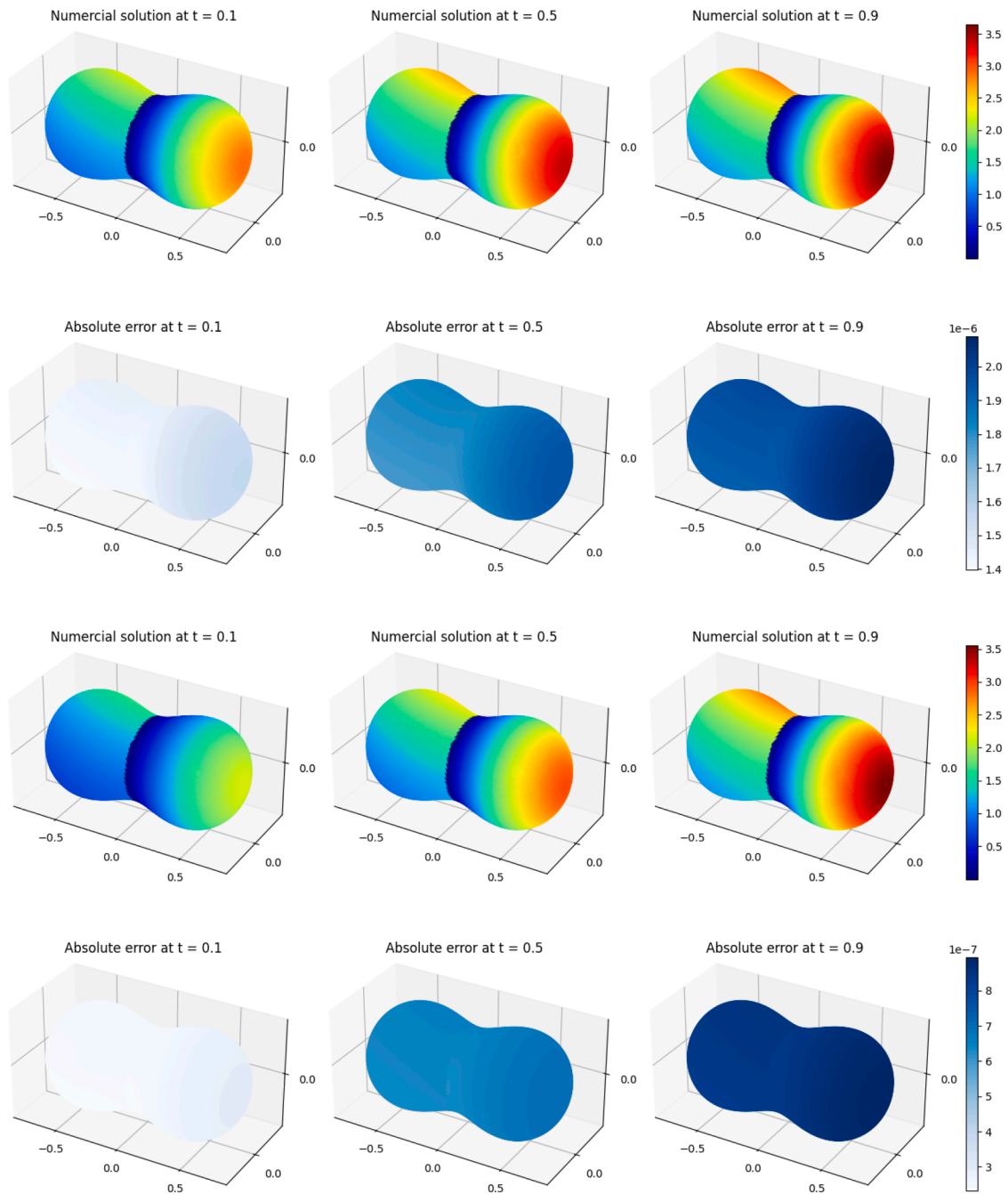
$$u = \begin{cases} (t^\alpha + t)zy \sin x, & \text{in } \tilde{\Gamma}^+, \\ (t^\alpha + t)zy \cos x, & \text{in } \tilde{\Gamma}^-. \end{cases}$$

On the sub-surface  $\tilde{\Gamma}^+$ , the diffusion coefficient  $\beta = \sin(x+y+z) + 2$  is used, whereas on  $\tilde{\Gamma}^-$ , it is defined by  $\beta = \cos(x+y+z) + 2$ .

We investigate how the hyperparameter  $N_t$ , ranging from 32 to 160, affects results. The quantities of training points are set to  $M_F = 4000$ ,  $M_\Lambda = 500$ ,  $M_{T_0} = 500$ . The accuracy of the SDC-PINN for this test case is shown in Table 8. Fig. 15 illustrates the network solutions and error distributions, while Fig. 16 depicts the evolution of the  $L^\infty$ -errors as time progresses. It is highlighted that increasing  $N_t$  enhances the precision of the numerical solution. This enhancement is mainly due to the decrease in  $\tau$  with rising  $N_t$ , which improves the accuracy of the approximation of  $\partial_t^\alpha U$ . Significantly, this phenomenon becomes increasingly evident as time progresses. Additionally, the results reveal that increasing  $N_t$  from 128 to 160 does not result in enhanced accuracy, indicating that the network's learning capacity has reached a plateau within this range. Consequently, a network configuration with  $N_t = 3$ ,  $N_u = 15$ ,  $N_t = 128$  is considered a more appropriate selection for the present study, achieving the smallest relative  $L^2$  error on the magnitude of  $\mathcal{O}(10^{-6})$ .

**Example 7.** In this case, problem (2.2) is considered on a hemispherical surface, which is expressed by

$$\phi = x^2 + y^2 + z^2 - 0.5^2, \quad z \geq 0.$$



**Fig. 14.** Example 5: the network solutions and error distributions. Here, we utilize the nonuniform L2-1<sub>σ</sub> scheme to approximate  $\partial_t^\alpha U$ . Top two rows:  $\alpha = 0.25$ ; Bottom two rows:  $\alpha = 0.75$ .

A cylindrical surface, which has a flower-like shape, is expressed by  $r(\Theta) = 0.05 \sin(5\Theta) + 0.2$ , where  $\Theta = \arcsin \frac{x}{\sqrt{x^2+y^2}}$ . The intersection curve between this cylindrical surface and  $\tilde{\Gamma}$  is defined as the interface  $\Lambda$ . We choose the exact solution as

$$u = \begin{cases} (1 + t^\alpha) e^{x+y+z}, & r^2(\Theta) - x^2 - y^2 > 0, \\ (1 + t^\alpha) \cos(x^2 + y^2 + z^2), & r^2(\Theta) - x^2 - y^2 < 0, \end{cases}$$

where  $\alpha = 0.25$ . For this test case, the sub-surface, described by

$$\tilde{\Gamma}^- = \{(x, y, z) | x^2 + y^2 + z^2 - 0.5^2 = 0, r^2(\Theta) - x^2 - y^2 < 0\},$$

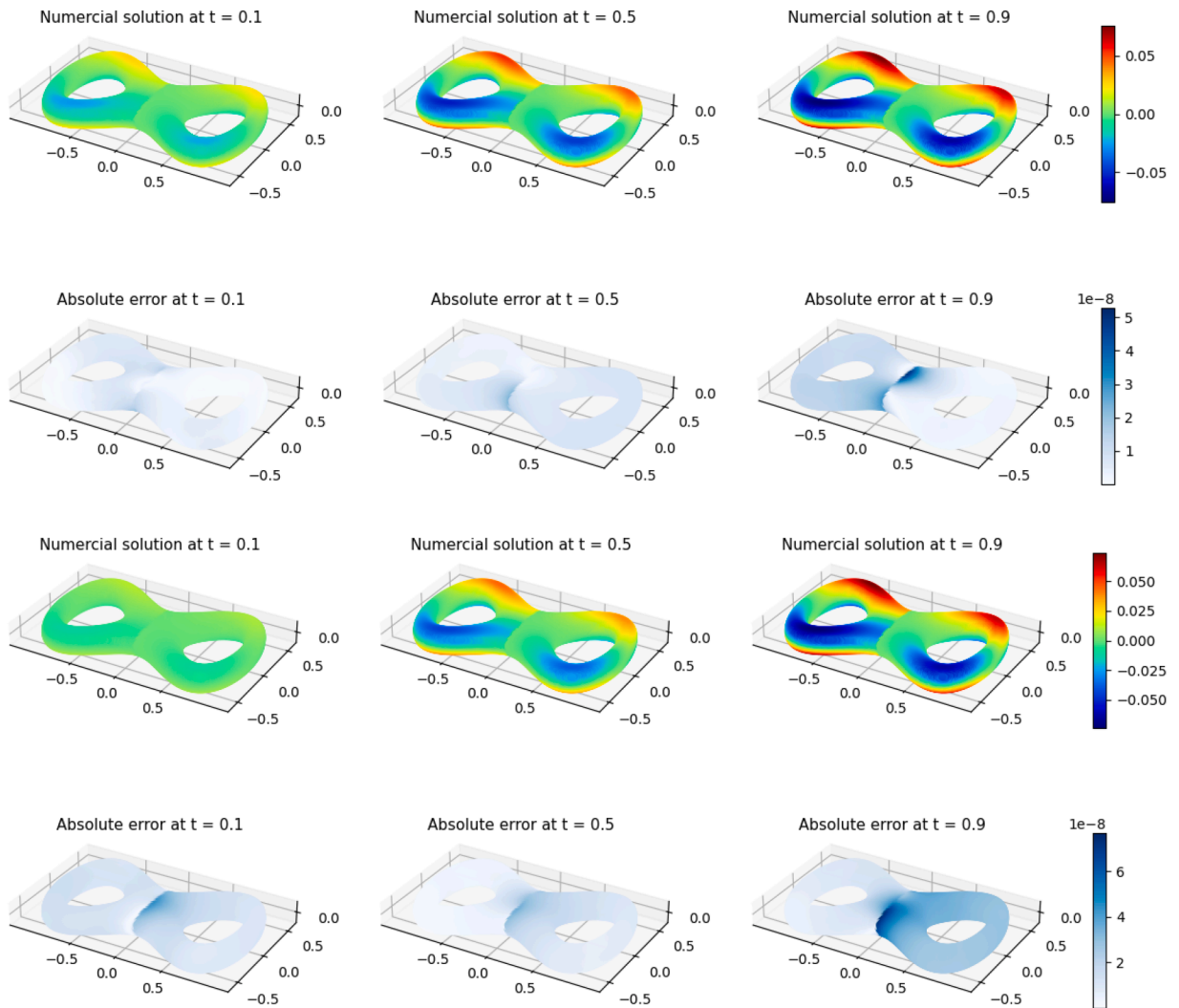


Fig. 15. Example 6: the network solutions and error distributions with  $N_t = 128$ . Top two rows:  $\alpha = 0, 25$ ; Bottom two rows:  $\alpha = 0.75$ .

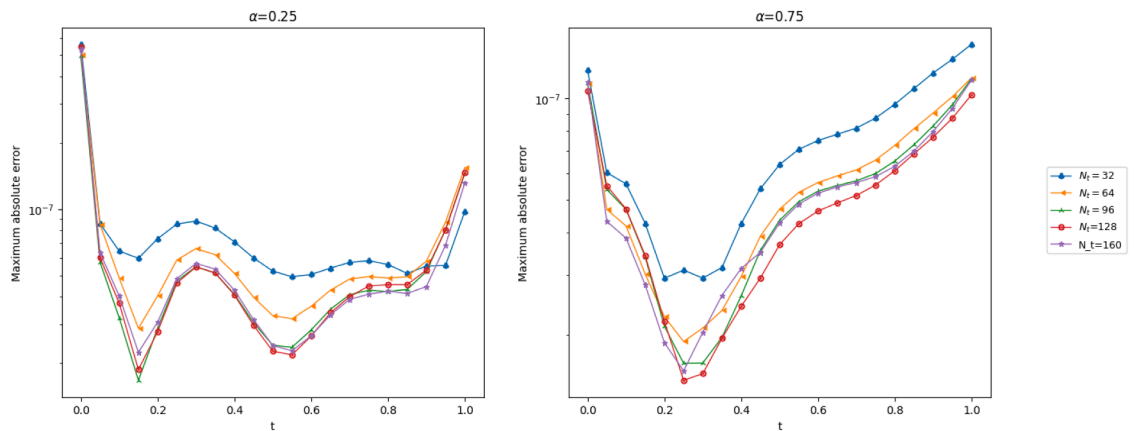


Fig. 16. Example 6: the maximum absolute errors over time.

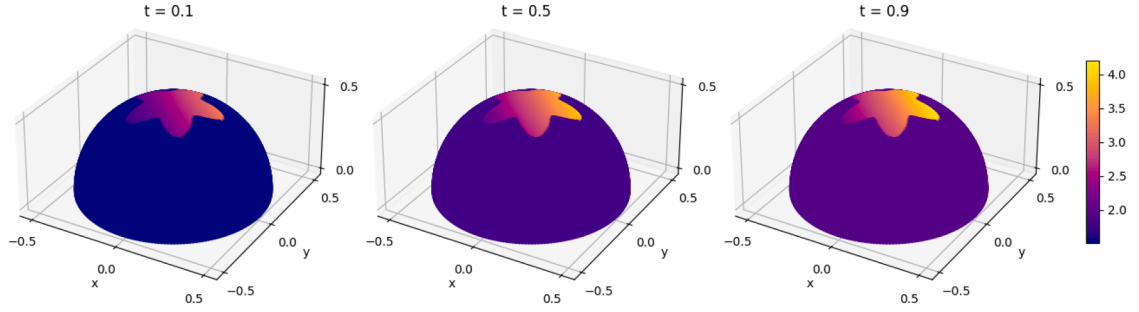


Fig. 17. Example 7: the exact solutions at times  $t = 0.1, 0.5$ , and  $0.9$ , respectively.

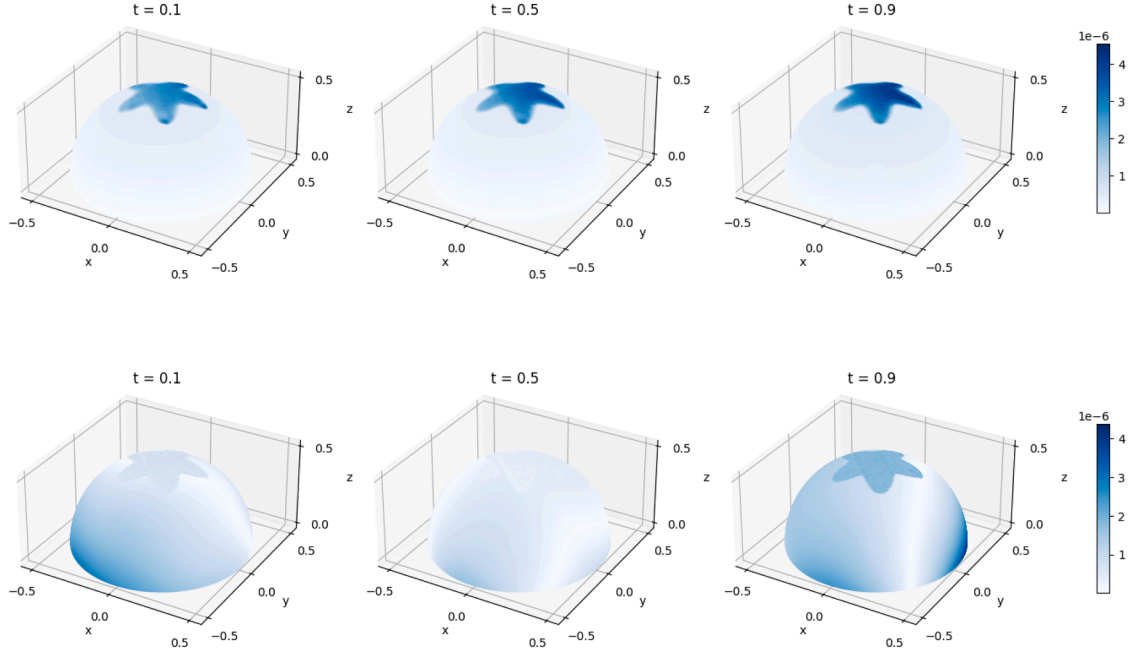


Fig. 18. Example 7: the error distributions at various instances, specifically at  $t = 0.1, 0.5$ , and  $0.9$ , respectively. Top:  $(\beta^+, \beta^-) = (0.001, 1)$ ; Bottom:  $(\beta^+, \beta^-) = (1000, 1)$ .

Table 9  
The accuracy in Example 7.

| $(\beta^+, \beta^-)$ | Relative $L^2$ -error | $L^\infty$ -error |
|----------------------|-----------------------|-------------------|
| (0.001,1)            | 3.99E-07              | 4.86E-06          |
| (0.01,1)             | 3.23E-07              | 2.71E-06          |
| (0.1,1)              | 2.36E-07              | 1.10E-06          |
| (1,1)                | 2.02E-07              | 6.26E-07          |
| (10,1)               | 1.98E-07              | 6.58E-07          |
| (100,1)              | 2.41E-07              | 4.79E-06          |
| (1000,1)             | 5.68E-07              | 1.13E-05          |

has a circular boundary, expressed as  $x^2 + y^2 = 0.5^2$ ,  $z = 0$ .

Unlike the instances previously studied, we now consider the treatment of Dirichlet boundary conditions. Specifically, during the training process, we can choose  $M_{\partial\tilde{\Gamma}}$  points from the set  $\mathcal{P}_{\partial\tilde{\Gamma}} = \{(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i) | \mathbf{x}_{\partial\tilde{\Gamma}}^i \in \partial\tilde{\Gamma}, t_{\partial\tilde{\Gamma}}^i \in (0, T], i = 1, 2, \dots, M_{\partial\tilde{\Gamma}}\}$  and add an extra term to the loss function such that

$$\frac{1}{M_{\partial\tilde{\Gamma}}} \sum_{(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i) \in \mathcal{P}_{\partial\tilde{\Gamma}}} L_{\partial\tilde{\Gamma}}(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i; \theta)^2 = \frac{1}{M_{\partial\tilde{\Gamma}}} \sum_{(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i) \in \mathcal{P}_{\partial\tilde{\Gamma}}} (\mathcal{U}(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i, \mathbf{I}(\mathbf{x}_{\partial\tilde{\Gamma}}^i), \mathcal{J}(t_{\partial\tilde{\Gamma}}^i)) - u_b(\mathbf{x}_{\partial\tilde{\Gamma}}^i, t_{\partial\tilde{\Gamma}}^i))^2.$$

The quantities of training points are set to  $M_{\tilde{\Gamma}} = 5000$ ,  $M_{\Lambda} = 1500$ ,  $M_{T_0} = 1000$ , and  $M_{\partial\tilde{\Gamma}} = 1500$ . The graph of the exact solution is illustrated in Fig. 17. To assess the effectiveness and robustness of the SDC-PINN method, its performance is evaluated across the

**Table 10**  
The accuracy in Example 8.

| $\alpha$ | Relative $L^2$ -error | $L^\infty$ -error |
|----------|-----------------------|-------------------|
| 0.1      | 7.63E-06              | 8.85E-05          |
| 0.3      | 3.92E-06              | 2.88E-05          |
| 0.5      | 1.52E-06              | 5.28E-06          |
| 0.7      | 2.03E-06              | 1.07E-05          |
| 0.9      | 2.19E-06              | 7.40E-06          |

**Table 11**  
The accuracy in Example 8 involving logarithmic initial singularity.

| $\alpha$ | Relative $L^2$ -error | $L^\infty$ -error |
|----------|-----------------------|-------------------|
| 0.1      | 2.68E-06              | 2.92E-05          |
| 0.3      | 9.26E-07              | 2.03E-06          |
| 0.5      | 6.32E-07              | 7.25E-08          |
| 0.7      | 6.10E-07              | 9.70E-08          |
| 0.9      | 5.06E-07              | 6.97E-08          |

following range of variable coefficients:

$$(\beta^+, \beta^-) = (0.001, 1), (0.01, 1), (0.1, 1), (1, 1), (10, 1), (100, 1), (1000, 1).$$

Notably, the high contrast ratio in the diffusion coefficient introduces stiffness into the interface problem. This stiffness leads to a significantly complex and highly non-convex mean squared error loss function, which results in an ill-conditioned loss landscape with numerous local minima, making training accurate solutions significantly challenging. The accuracy is shown in Table 9, while the distributions of error are depicted in Fig. 18. One can clearly see that the error is mainly concentrated in  $\tilde{\Gamma}^+$ , due to the complex geometry of the interface. Nonetheless, it can still reach a magnitude of approximately  $\mathcal{O}(10^{-5})$ . Notably, the SDC-PINN consistently achieves a final relative  $L^2$ -error on the order of  $\mathcal{O}(10^{-7})$  for all cases, showcasing its effectiveness in resolving problems (2.2) with discontinuous and high contrast coefficients.

**Example 8.** In this example, we demonstrate the capability of the proposed SDC-PINN to address problem (2.2) with an intricate surface and a complex interface, considering a range of  $\alpha$  values, specifically from 0.1 to 0.9. The curved surface is expressed by

$$\phi = (1 + 4x^2)(1 + 4y^2)(1 + 4z^2) + 64xyz + 4x^2 + 4y^2 + 4z^2 - 3.$$

We choose the interface as the intersection curve between the flower shape cylindrical surface and the upper half of  $\tilde{\Gamma}$ , i.e.  $\Lambda = \{\mathbf{x} | r^2(\Theta) - x^2 - y^2 = 0\} \cap \{\mathbf{x} | \phi(\mathbf{x}) = 0, z \geq 0\}$ . Using R-functions [22,24], the interface  $\Lambda$  can be expressed by

$$\omega_1 = r^2(\Theta) - x^2 - y^2 = 0, \quad \omega_2 = z, \quad \varphi = \omega_1 + \omega_2 - \sqrt{\omega_1^2 + \omega_2^2}.$$

The sub-surface  $\tilde{\Gamma}^+$  is defined as the section of  $\tilde{\Gamma}$  where  $\varphi(\mathbf{x}) > 0$ , while  $\tilde{\Gamma}^-$  corresponds to the section of  $\tilde{\Gamma}$  where  $\varphi(\mathbf{x}) < 0$ . We choose the analytic solution as

$$u = \begin{cases} (t^\alpha + 1)(yx \cos(4\pi z^2) - 0.1), & \text{in } \tilde{\Gamma}^-, \\ (t^\alpha + 1)(yx \sin(4\pi z^2) + 0.1), & \text{in } \tilde{\Gamma}^+. \end{cases}$$

On the sub-surface  $\tilde{\Gamma}^+$ , the diffusion coefficient  $\beta = \sin(x + y + z) + 2$  is used, whereas on  $\tilde{\Gamma}^-$ , it is defined by  $\beta = \cos(x + y + z) + 2$ . In contrast to the preceding example, for this specific case, we define the reaction coefficient  $\kappa$  to be  $\cos(x + y + z) + 2$  on the sub-surface  $\tilde{\Gamma}^+$ , whereas on the sub-surface  $\tilde{\Gamma}^-$ , we set it to  $\sin(x + y + z) + 2$ .

We assign the values of  $M_{\tilde{\Gamma}} = 4500$ ,  $M_\Lambda = 1000$ , and  $M_{T_0} = 800$ . It is important to emphasize that the small  $\alpha$  values result in a strong initial singularity, which presents a considerable challenge for the neural network in accurately approximating the solution. Additionally, the intricate geometries of both the surface and interface may vary significantly with curvature, which can lead to an initial unbalanced loss value between each training point, thereby substantially complicating the training process. The accuracy of the SDC-PINN is presented in Table 10. Fig. 19 illustrates the network solutions alongside the error distributions. Notably, the SDC-PINN achieves a final relative  $L^2$ -error approximately on the magnitude of  $\mathcal{O}(10^{-6})$ , even when  $\alpha$  is set to 0.1. This finding suggests that the SDC-PINN is adept at handling challenges posed by complex surfaces and interfaces, even with small values of  $\alpha$ . Furthermore, the trajectories of  $\xi$  during the training process are plotted in Fig. 20. It is observed that the trainable parameter  $\xi$  rapidly converges to a value close to  $\alpha$ , thereby enabling the network to model the initial singularity precisely.

To further assess the robustness and flexibility of the SDC-PINN, we investigate its performance on problems with initial singularities that deviate from simple power-law behavior.

First, we consider a solution featuring a logarithmic initial singularity term:

$$u = \begin{cases} (\ln(1+t))^\alpha (0.6yx \cos(z^2) - 0.1), & \text{in } \tilde{\Gamma}^-, \\ (\ln(1+t))^\alpha (0.6yx \sin(z^2) + 0.1), & \text{in } \tilde{\Gamma}^+. \end{cases}$$

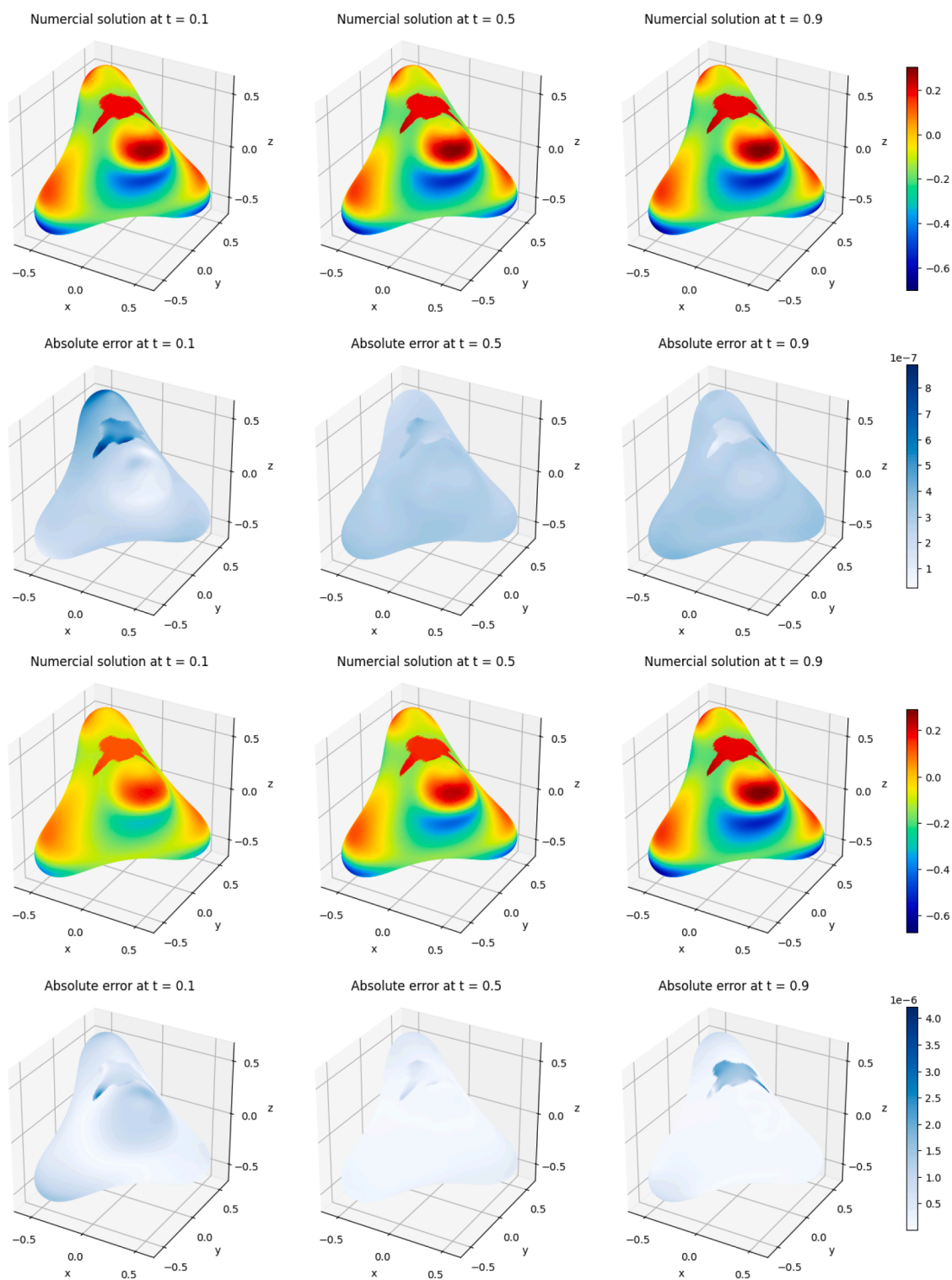


Fig. 19. Example 8: the network solutions and error distributions. Top two rows:  $\alpha = 0.1$ ; Bottom two rows:  $\alpha = 0.9$ .

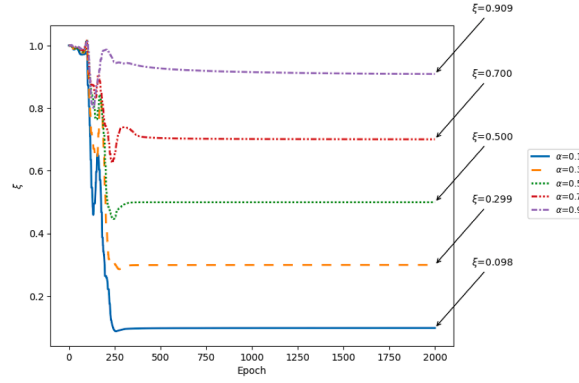


Fig. 20. Example 8: the trajectory of  $\xi$  during the iterative training process.

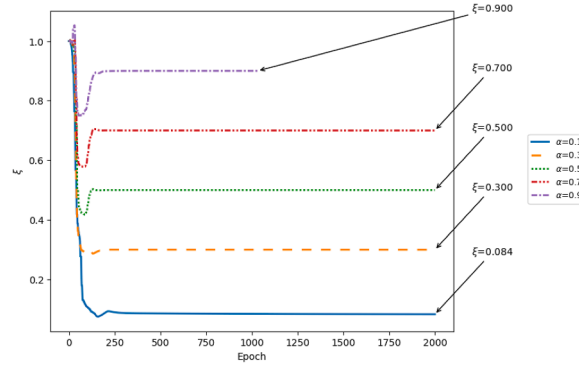


Fig. 21. Example 8: the evolution of  $\xi$  for the logarithmic initial singularity case.

Table 12

Accuracy in Example 8 for the non-power-law initial singularity case.

| $\alpha$ | Relative $L^2$ -error | $L^\infty$ -error |
|----------|-----------------------|-------------------|
| 0.1      | 6.64E-06              | 1.44E-05          |
| 0.3      | 5.73E-06              | 2.02E-05          |
| 0.5      | 5.05E-06              | 1.33E-05          |
| 0.7      | 7.95E-06              | 1.78E-05          |
| 0.9      | 1.16E-05              | 1.79E-05          |

Near  $t = 0$ , the Taylor expansion

$$\ln(1+t) = t - \frac{t^2}{2} + \frac{t^3}{3} - \dots,$$

shows that the leading behavior is equivalent to  $t^\alpha$ , but the higher-order terms introduce additional complexity. For this test, we set the network hyperparameters to  $N_l = 3$ ,  $N_u = 20$ ,  $N_f = 160$ , and  $N_\xi = 1$ . As demonstrated in Fig. 21, the learned exponent  $\xi$  rapidly converges to a value close to  $\alpha$  for all tested cases. This successful identification of the dominant singularity enables the model to achieve excellent precision, with relative  $L^2$  and  $L^\infty$  errors consistently on the order of  $10^{-5}$  or lower, as detailed in Table 11.

Next, we evaluate the model's capability to handle a non-power-law singularity. We define the solution as:

$$u = \begin{cases} (e^t - 1) \ln(t) (0.5yx \cos(z^2) - 0.1), & \text{in } \tilde{\Gamma}^-, \\ (e^t - 1) \ln(t) (yx \sin(z^2) + 0.1), & \text{in } \tilde{\Gamma}^+. \end{cases}$$

To provide the network with greater flexibility to capture this complex behavior, we increase the number of learned exponents to  $N_\xi = 3$ , with other hyperparameters set to  $N_l = 3$ ,  $N_u = 20$ , and  $N_f = 160$ . The resulting error metrics are presented in Table 12. Despite the challenging nature of the  $\ln(t)$  term, the SDC-PINN maintains high accuracy. Fig. 22 reveals that the minimum value among the learned exponents,  $\min(\xi_i)$ , consistently converges to a value around 0.88, which enables the network to effectively approximate the non-power-law initial singularity, showcasing its adaptability and robustness.

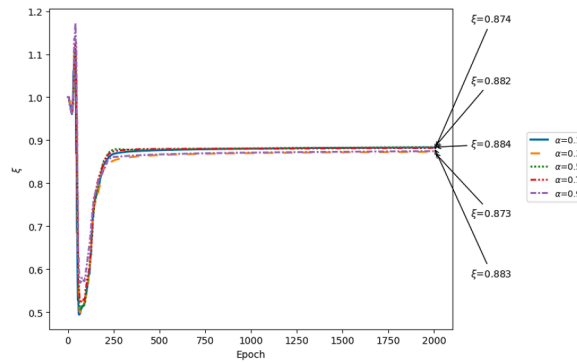


Fig. 22. Example 8: the evolution of the minimum  $\xi$  for the non-power type initial singularity case.

**Example 9.** The final example serves to demonstrate the capability of the proposed SDC-PINN in solving time-fractional PDEs that exhibit multi-singularities. We consider the following surface time-fractional delay equations with interface:

$$\begin{cases} \partial_t^\alpha u(\mathbf{x}, t) - \nabla_{\tilde{\Gamma}} \cdot (\beta \nabla_{\tilde{\Gamma}} u(\mathbf{x}, t)) + \kappa u(\mathbf{x}, t) + \gamma u(\mathbf{x}, t - \mu) = f(\mathbf{x}, t), & \mathbf{x} \in \tilde{\Gamma}^+ \cup \tilde{\Gamma}^-, t \in (0, T], & (4.1a) \\ [u]|_\Lambda = \lambda(\mathbf{x}_\Lambda, t), & \mathbf{x}_\Lambda \in \Lambda, t \in (0, T], & (4.1b) \\ [\beta \nabla_{\tilde{\Gamma}} u \cdot \mathbf{n}_\Lambda]|_\Lambda = \rho(\mathbf{x}_\Lambda, t), & \mathbf{x}_\Lambda \in \Lambda, t \in (0, T], & (4.1c) \\ u(\mathbf{x}, t) = \psi(\mathbf{x}, t), & \mathbf{x} \in \tilde{\Gamma}^+ \cup \tilde{\Gamma}^-, t \in [-\mu, 0], & (4.1d) \end{cases}$$

where  $\gamma$  is a given smooth function and  $\mu$  represents the constant delay parameter. The solution and its temporal derivatives satisfy the following regularity condition [34]:

$$|\partial_t^\ell u(\cdot, t)| \leq C(1 + (t - (i-1)\mu)^{\alpha-\ell}), \quad \ell = 0, 1, 2, \quad (4.2)$$

where  $i = 1, 2, \dots, M$  with  $M = \lceil T/\mu \rceil$ .

To address the multi-singularities inherent in the problem (4.1), three key adaptations are made. First, to address the discontinuity in the temporal derivatives, a time-marching strategy is integrated with the SDC-PINN framework [35]. The time domain is divided into  $M$  intervals:

$$(0, \mu], (\mu, 2\mu], \dots, ((M-1)\mu, T], \quad (4.3)$$

and a distinct neural network  $\mathcal{U}_i$  is trained for each interval. For the first interval, the problem (4.1) is solved using the standard SDC-PINN, with  $\psi(\mathbf{x}, 0)$  serving as the initial condition. For each subsequent interval, the initial condition is prescribed by the final state of the previous network. Second, the singularity-capturing feature in the  $i$ -th interval is reset to  $\mathcal{Z}_i = \mathcal{J}_i(t) = (t - (i-1)\mu)^\xi$  to handle the singularity at the start of the interval. Lastly, the PDE (4.1a) is reformulated to isolate the history integral over the current interval:

$$\frac{1}{\Gamma(1-\alpha)} \int_{(i-1)\mu}^t (t-s)^{-\alpha} \partial_s u(\mathbf{x}, s) ds - \nabla_{\tilde{\Gamma}} \cdot (\beta \nabla_{\tilde{\Gamma}} u(\mathbf{x}, t)) + \kappa u(\mathbf{x}, t) = \tilde{f}, \quad (4.4)$$

where  $\tilde{f} = f(\mathbf{x}, t) - \frac{1}{\Gamma(1-\alpha)} \int_0^{(i-1)\mu} (t-s)^{-\alpha} \partial_s u(\mathbf{x}, s) ds - \gamma u(\mathbf{x}, t - \mu)$  is evaluated using the already-trained networks.

To accurately evaluate the integral in (4.4), especially near  $t = (j-1)\mu$ ,  $j = 1, 2, \dots, i$ , a specialized discretization scheme is required. Following [34], we generate the following auxiliary points

$$t_{j,k} = \begin{cases} \mu \left(\frac{k}{N_t}\right)^r, & j = 1, 0 \leq k \leq N_t, \\ \mu \left(\frac{k}{N_t}\right)^r + (j-1)\mu, & 2 \leq j \leq i-1, 1 \leq k \leq N_t, \\ (t - (i-1)\mu) \left(\frac{k}{N_t}\right)^r + (i-1)\mu, & j = i, 1 \leq k \leq N_t. \end{cases} \quad (4.5)$$

This allows the history and current interval integrals to be approximated as:

$$\begin{aligned} \frac{1}{\Gamma(1-\alpha)} \int_0^{(i-1)\mu} (t-s)^{-\alpha} \partial_s u(\mathbf{x}, s) ds &\approx \sum_{k=0}^{N_t} \mathcal{A}_{1,k} \mathcal{U}_1(\mathbf{x}, t_{1,k}, \mathcal{I}(\mathbf{x}), \mathcal{J}_1(t_{1,k})) + \sum_{j=2}^{i-1} \sum_{k=1}^{N_t} \mathcal{A}_{j,k} \mathcal{U}_j(\mathbf{x}, t_{j,k}, \mathcal{I}(\mathbf{x}), \mathcal{J}_j(t_{j,k})), \\ \frac{1}{\Gamma(1-\alpha)} \int_{(i-1)\mu}^t (t-s)^{-\alpha} \partial_s u(\mathbf{x}, s) ds &\approx \sum_{k=1}^{N_t} \mathcal{A}_{i,k} u(\mathbf{x}, t_{i,k}). \end{aligned} \quad (4.6)$$

The coefficients  $\mathcal{A}_{j,k}$  are determined using the nonuniform L1 scheme (2.8). Consequently, the residual error of the PDE for the  $i$ -th interval,  $L_{f,i}$  is defined as:

$$L_{f,i} = \sum_{k=1}^{N_i} \mathcal{A}_{i,k} \mathcal{U}_i(\mathbf{x}, t_{i,k}, \mathcal{I}(\mathbf{x}), \mathcal{J}_i(t_{i,k})) - \nabla_{\tilde{\Gamma}} \cdot \left( \beta \nabla_{\tilde{\Gamma}} \mathcal{U}_i(\mathbf{x}, t, \mathcal{I}(\mathbf{x}), \mathcal{J}_i(t)) \right) + \kappa \mathcal{U}_i(\mathbf{x}, t, \mathcal{I}(\mathbf{x}), \mathcal{J}_i(t)) - \tilde{f}. \quad (4.7)$$

We now consider the problem (4.1) defined on the following surface

$$\phi = x^2 + y^2 + 3z^2(z^2 - 0.99) = 0.02, \quad z \in [-0.3, 0.3].$$

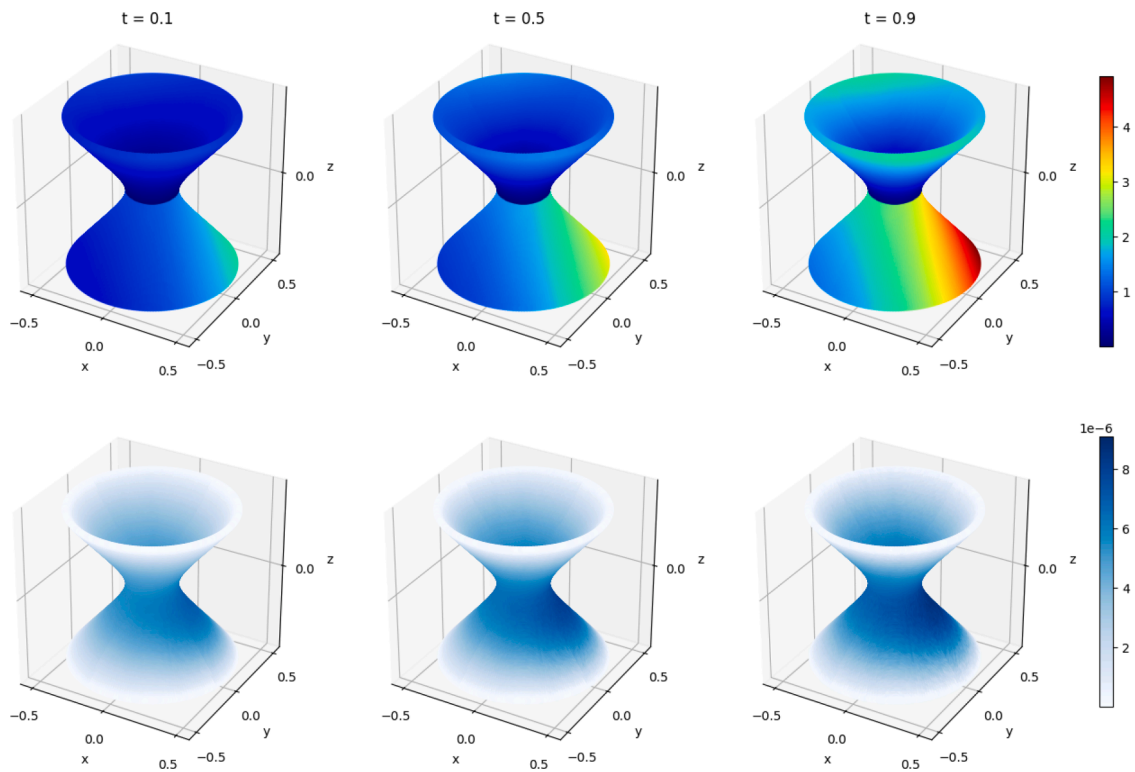


Fig. 23. Example 9: the network solutions and error distributions with  $\alpha = 0.2$ .

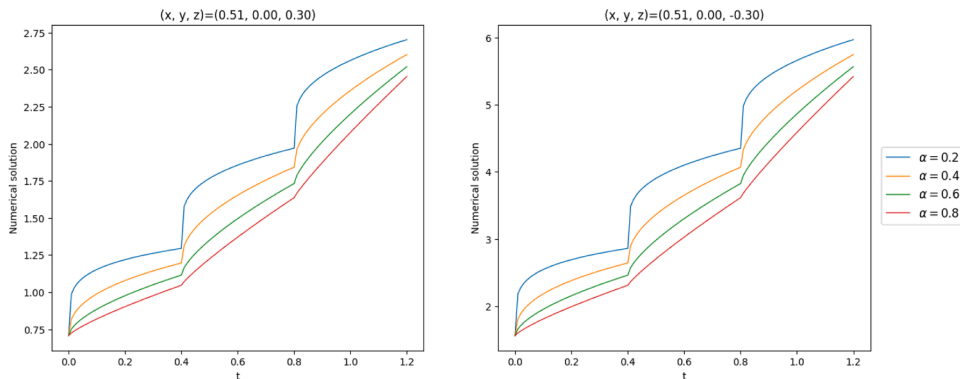


Fig. 24. Example 9: time evolution of the numerical solutions at two points:  $\mathbf{x} = (0.51, 0, 0.3) \in \tilde{\Gamma}^+$  (left) and  $\mathbf{x} = (0.51, 0, -0.3) \in \tilde{\Gamma}^-$  (right).

**Table 13**  
Accuracy in Example 9.

| $\alpha$ | Relative $L^2$ -error | $L^\infty$ -error |
|----------|-----------------------|-------------------|
| 0.2      | 1.14E-06              | 1.02E-05          |
| 0.4      | 1.80E-06              | 6.56E-06          |
| 0.6      | 2.81E-06              | 8.00E-06          |
| 0.8      | 3.71E-06              | 1.00E-05          |

The interface is the circle  $x^2 + y^2 = 0.02$  at  $z = 0$ , which separates  $\tilde{\Gamma}$  into an upper subdomain,  $\tilde{\Gamma}^+$ , and a lower subdomain,  $\tilde{\Gamma}^-$ . The analytic solution for the  $i$ -th time interval,  $(0.4(i-1), 0.4i]$  for  $i = 1, 2, 3$ , is given by:

$$u = \begin{cases} \left(1 + \sum_{j=1}^i (t - 0.4(j-1))^\alpha\right) \sin(z) e^{\cos(y+x)}, & \text{in } \tilde{\Gamma}^+, \\ \left(1 + \sum_{j=1}^i (t - 0.4(j-1))^\alpha\right) \cos(z) e^{\sin(y+x)}, & \text{in } \tilde{\Gamma}^-. \end{cases}$$

The delay condition is defined as

$$\psi = \begin{cases} \sin(z) e^{\cos(y+x)}, & \text{in } \tilde{\Gamma}^+, \\ \cos(z) e^{\sin(y+x)}, & \text{in } \tilde{\Gamma}^-. \end{cases}$$

The diffusion coefficient is set to  $\beta = \sin(x + y + z) + 2$  in  $\tilde{\Gamma}^+$  and  $\beta = \cos(x + y + z) + 2$  in  $\tilde{\Gamma}^-$ . The coefficients  $\kappa$  and  $\gamma$  are both set to 1.

The numbers of training points are set as  $M_{\tilde{\Gamma}} = 3000$ ,  $M_{\Lambda} = M_{T_0} = M_{\partial\tilde{\Gamma}} = 500$ . The shape parameter of the graded mesh  $r$  is set to  $(2 - \alpha)/\alpha$ . The numerical results are summarized in Table 13. Visualizations of the predicted solutions and the error distributions are provided in Fig. 23. The cross-sectional views of the network solutions are depicted in Fig. 24. One can clearly see that the proposed SDC-PINN effectively captures the singularities at  $t = 0, 0.4$ , and  $0.8$ , and consistently achieves errors on the magnitude of  $\mathcal{O}(10^{-5})$ . This performance confirms the method's effectiveness and robustness for solving time-fractional PDEs with multiple singularities.

## 5. Conclusion

In this paper, we propose the SDC-PINN as an innovative approach to address time-fractional diffusion problems characterized by initial singularities and interfaces on complex curved surfaces. The SDC-PINN integrates both discontinuity-capturing and initial singularity-capturing features into the neural network input. Furthermore, to effectively resolve the initial singularity, we employ a temporal graded mesh and a nonuniform L2-1 $_{\sigma}$  scheme for discretizing the Caputo fractional derivative. The solution offered by the neural network retains the inherent properties of the original solution, which can demonstrate discontinuities not only in the solution itself but also in its derivatives, and may also involve an initial singularity. We utilize the ResNet architecture as the primary framework for the network and apply the geodesic acceleration Levenberg-Marquardt method for network optimization. Numerous results from numerical experiments demonstrate that the proposed SDC-PINN achieves high levels of accuracy, even in scenarios involving complex curved surfaces and interfaces, high-contrast coefficients, intricate initial singularities, and multiple singularities. The comparative analysis confirms that the proposed SDC-PINN yields significant improvements in both performance and accuracy over prior neural network methods.

The scalability of the SDC-PINN, while positive, appears to be approaching a bottleneck, which may limit the attainment of full machine accuracy with the current architecture. To the best of our knowledge, transcending this limitation could be achieved through the implementation of the multi-grade deep learning strategy [38]. A formal investigation into such architectures constitutes a promising direction for future research, but is beyond the purview of the current paper. The scope of the present work is confined to surface time-fractional diffusion problems with interfaces. Future research directions may include the application of this approach to various practical applications.

## Data availability

No data was used for the research described in the article.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

The work is supported by the National Nature Science Foundation of China (12371418), Guangdong Natural Science Foundation (2024A1515010694), and Guangdong Province Key Laboratory of Computational Science at the Sun Yat-sen University.

## Appendix A. Mathematical notations

For clarity, the main mathematical notations used in this paper are summarized in [Table A.1](#).

**Table A.1**  
Definition of common mathematical notations used in the paper.

| Notation                                             | Definition                                                                                      |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <i>PDE Formulation</i>                               |                                                                                                 |
| $u$                                                  | Analytic solution of the PDE.                                                                   |
| $\beta, \kappa$                                      | Diffusion and reaction coefficients, respectively.                                              |
| $\alpha$                                             | Order of the Caputo fractional derivative.                                                      |
| $\partial_t^\alpha u, {}^C D_t^\alpha u$             | Caputo fractional derivative of order $\alpha$ .                                                |
| $\Gamma(\cdot)$                                      | Gamma function.                                                                                 |
| $\tilde{\Gamma}$                                     | Curved surface, representing the computational domain.                                          |
| $\Lambda$                                            | A closed curve on $\tilde{\Gamma}$ , representing the interface.                                |
| $\phi, \varphi$                                      | Level-set functions for $\tilde{\Gamma}$ and $\Lambda$ , respectively.                          |
| $\nabla_{\tilde{\Gamma}}, \nabla_{\tilde{\Gamma}^*}$ | Surface gradient and surface divergence operators on $\tilde{\Gamma}$ , respectively.           |
| <i>Discretization Schemes</i>                        |                                                                                                 |
| $r$                                                  | Shape parameter for the graded mesh.                                                            |
| $D_N^\alpha u^n$                                     | L1 approximation of the $\partial_t^\alpha u(\cdot, t_n)$ .                                     |
| $a_{n,k}$                                            | Coefficients of the L1 scheme for $n$ time steps.                                               |
| $\delta_{t_{n-\sigma}}^\alpha u$                     | L2-1 $_\sigma$ approximation of the $\partial_t^\alpha u(\cdot, t_{n-\sigma})$ .                |
| $C_k^{n,\alpha}$                                     | Coefficients of the L2-1 $_\sigma$ scheme for $n$ time steps.                                   |
| <i>Neural Network</i>                                |                                                                                                 |
| $\mathcal{U}^*$                                      | Neural network solution.                                                                        |
| $\mathcal{Y}$                                        | Discontinuity-capturing feature.                                                                |
| $\mathcal{Z}$                                        | Singularity-capturing feature.                                                                  |
| $\xi, \tilde{\xi}$                                   | Trainable vector within $\mathcal{Z}$ , related by the transformation $\xi = e^{\tilde{\xi}}$ . |
| $\theta$                                             | Set of all trainable parameters in the neural network.                                          |
| $L_f$                                                | Residual error of PDE.                                                                          |
| $L_{\Lambda_D}, L_{\Lambda_N}$                       | Residual error of jump conditions.                                                              |
| $L_{T_0}$                                            | Residual error of initial condition.                                                            |
| $L_{\partial\tilde{\Gamma}}$                         | Residual error of boundary condition.                                                           |
| $N_{\tilde{\xi}}$                                    | Length of the vector $\tilde{\xi}$ .                                                            |
| $N_t$                                                | Number of auxiliary points for calculating $\delta_t^\alpha \mathcal{U}$ .                      |
| $N_l$                                                | Number of residual blocks in neural network.                                                    |
| $N_u$                                                | Number of units per hidden layer.                                                               |
| $M_{\tilde{\Gamma}}$                                 | Number of training points for $L_f$ .                                                           |
| $M_\Lambda$                                          | Number of training points for $L_{\Lambda_D}$ and $L_{\Lambda_N}$ .                             |
| $M_{T_0}$                                            | Number of training points for $L_{T_0}$ .                                                       |
| $M_{\partial\tilde{\Gamma}}$                         | Number of training points for $L_{\partial\tilde{\Gamma}}$ .                                    |

## References

- [1] D.A. Benson, S.W. Wheatcraft, M.M. Meerschaert, Application of a fractional advection-dispersion equation, *Water Resour. Res.* 36 (6) (2000) 1403–1412.
- [2] A.H. Bhrawy, M.M. Tharwat, A. Yildirim, A new formula for fractional integrals of chebyshev polynomials: application for solving multi-term fractional differential equations, *Appl. Math. Model.* 37 (6) (2013) 4245–4252.
- [3] W. Bu, Y. Tang, J. Yang, Galerkin finite element method for two-dimensional riesz space fractional diffusion equations, *J. Comput. Phys.* 276 (2014) 26–38.
- [4] Y. Chen, Q. Li, H. Yi, Y. Huang, Immersed finite element method for time fractional diffusion problems with discontinuous coefficients, *Comput. Math. Appl.* 128 (2022) 121–129.
- [5] W. Deng, Finite element method for the space and time fractional fokker-planck equation, *SIAM J. Numer. Anal.* 47 (1) (2009) 204–226.
- [6] G.-H. Gao, Z.-Z. Sun, A compact finite difference scheme for the fractional sub-diffusion equations, *J. Comput. Phys.* 230 (3) (2011) 586–595.
- [7] L. Guo, H. Wu, X. Yu, T. Zhou, Monte Carlo fpinns: Deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations, *Comput. Methods Appl. Mech. Eng.* 400 (2022) 115523.
- [8] C. He, X. Hu, L. Mu, A mesh-free method using piecewise deep neural network for elliptic interface problems, *J. Comput. Appl. Math.* 412 (2022) 114358.
- [9] X.K. He, S. Zhang, J. Ren, Sun, Deep residual learning for image recognition, in: *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 770–778.
- [10] W.-F. Hu, T.-S. Lin, M.-C. Lai, A discontinuity capturing shallow neural network for elliptic interface problems, *J. Comput. Phys.* 469 (2022) 111576.
- [11] G. Lewicki, G. Marino, Approximation by superpositions of a sigmoidal function, *Zeitschrift Fur Anal. Und Ihre Anwendungen* 22 (2003) 463–470.
- [12] C. Li, F. Zeng, Finite difference methods for fractional differential equations, *Int. J. Bifurcation Chaos* 22 (04) (2012) 1230014.
- [13] H.-L. Liao, D. Li, J. Zhang, Sharp error estimate of the nonuniform L1 formula for linear reaction-subdiffusion equations, *SIAM J. Numer. Anal.* 56 (2) (2018) 1112–1133.
- [14] W.H.L. Liao, J. Mclean, Zhang, A second-order scheme with nonuniform time steps for a linear reaction-subdiffusion problem, *Commun. Comput. Phys.* 30 (2021) 567–601.

- [15] F. Mainardi, Fractional Calculus and Waves in Linear Viscoelasticity: An Introduction to Mathematical Models, World Scientific, 2022.
- [16] G. Pang, L. Lu, G.E. Karniadakis, fpinns: Fractional physics-informed neural networks, *SIAM J. Sci. Comput.* 41 (4) (2019) A2603–A2626.
- [17] P.-O. Persson, G. Strang, A simple mesh generator in Matlab, *SIAM Rev.* 46 (2) (2004) 329–345.
- [18] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [19] X.H. Ren, R. Meng, J. Liu, Y. Hou, Yu, A class of improved fractional physics informed neural networks, *Neurocomputing* 562 (2023) 126890.
- [20] S. Roy, C. Annavarapu, P. Roy, A.K. Sarma, Adaptive interface-pinns (AdaI-PINNs): an efficient physics-informed neural networks framework for interface problems, [arXiv:2406.04626](https://arxiv.org/abs/2406.04626), 2024.
- [21] A.K. Sarma, S. Roy, C. Annavarapu, P. Roy, S. Jagannathan, Interface PINNs (I-PINNs): A physics-informed neural networks framework for interface problems, *Comput. Methods Appl. Mech. Eng.* 429 (2024) 117135.
- [22] V. Shapiro, Semi-analytic geometry with r-functions, *Acta Numer.* 16 (2007) 239–303.
- [23] M. Stynes, O. Eugene, J. Riordan, G. Luis, Error analysis of a finite difference method on graded meshes for a time-fractional diffusion equation, *SIAM J. Numer. Anal.* 55 (2) (2017) 1057–1079.
- [24] N. Sukumar, A. Srivastava, Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks, *Comput. Methods Appl. Mech. Eng.* 389 (2022) 114333.
- [25] M.K. Transtrum, J.P. Sethna, Improvements to the Levenberg-Marquardt algorithm for nonlinear least-squares minimization, [arXiv:1201.5885](https://arxiv.org/abs/1201.5885), 2012.
- [26] Y.-H. Tseng, T.-S. Lin, W.-F. Hu, M.-C. Lai, A cusp-capturing pinn for elliptic interface problems, *J. Comput. Phys.* 491 (2023) 112359.
- [27] S. Wu, B. Lu, INN: Interfaced neural networks as an accessible meshless approach for solving interface PDE problems, *J. Comput. Phys.* 470 (2022) 111588.
- [28] Y.-F. Pu, Fractional differential analysis for texture of digital image, *Journal of Algorithms & Computational Technology* 1 (3) (2007) 357–380.
- [29] F. Zeng, F. Liu, C. Li, K. Burrage, I. Turner, V. Anh, A crank-nicolson adi spectral method for a two-dimensional riesz space fractional nonlinear reaction-diffusion equation, *SIAM J. Numer. Anal.* 52 (6) (2014) 2599–2622.
- [30] H. Li, H. Fan, Z. Tan, Two novel discontinuity-removing PINNs for solving variable coefficient elliptic interface problems on curved surfaces, *Comput. Methods Appl. Mech. Eng.* 435 (2025) 117637.
- [31] H. Fan, Z. Tan, Novel and general discontinuity-removing PINNs for elliptic interface problems, *J. Comput. Phys.* 529 (2025) 113861.
- [32] J. Ying, J. Hu, Z. Shi, J. Li, An accurate and efficient continuity-preserved method based on randomized neural networks for elliptic interface problems, *SIAM J. Sci. Comput.* 46 (2024) C633–C657.
- [33] Y. Liang, Q. Zhang, S. Zeng, A piecewise extreme learning machine for interface problems, *Math. Comput. Simul.* 227 (2025) 303–321.
- [34] S. Patel, S. Santra, R. Behera, An efficient hybrid numerical approach for time-fractional sub-diffusion equations with multi-singularities, *Numer. Algorithms*, 2025, 1–38. <https://doi.org/10.1007/s11075-025-02122-9>.
- [35] C.L. Wight, J. Zhao, Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks, *Commun. Comput. Phys.* 29 (2021) 930–954.
- [36] T. Helin, M. Lassas, L. Ylinen, Z. Zhang, Inverse problems for heat equation and space-time fractional diffusion equation with one measurement, *J. Differ. Equations* 269 (2020) 7498–7528.
- [37] Z. Tang, Z. Fu, H. Sun, X. Liu, An efficient localized collocation solver for anomalous diffusion on surfaces, *Fract. Calc. Appl. Anal.* 24 (2021) 865–894.
- [38] Y. Wang, C.-Y. Lai, Multi-stage neural networks: Function approximator of machine precision, *J. Comput. Phys.* 504 (2024) 112865.